

Administración de procesos

Proceso: definición y características

Una cuestión que surge al hablar de los sistemas operativos es cómo llamar a todas las actividades de la CPU. Los primeros computadores eran sistemas por lotes que ejecutaban trabajos, seguidos por la aparición de sistemas de tiempo compartido que ejecutaban programas de usuario, o tareas. Incluso en un sistema monoprocesador, un usuario puede ejecutar varios programas a la vez: un procesador de textos, un navegador web y un paquete de correo electrónico. E incluso si un computador sólo puede ejecutar un programa a la vez, como en un dispositivo integrado que no admite la multitarea, el sistema operativo puede necesitar apoyar sus propias actividades internas programadas, como la gestión de la memoria. En muchos aspectos, todas estas actividades son similares, por lo que las llamamos *procesos*.

Aunque personalmente preferimos el término más contemporáneo de proceso, el término *trabajo* tiene un significado histórico, ya que gran parte de la teoría y la terminología de los sistemas operativos se desarrolló durante una época en la que la principal actividad de los sistemas operativos era el procesamiento de trabajos. Por lo tanto, en algunos casos apropiados utilizamos trabajo cuando describimos el papel del sistema operativo. Por ejemplo, sería engañoso evitar el uso de términos comúnmente aceptados que incluyen la palabra trabajo (como programación de trabajos) simplemente porque el proceso ha sustituido al trabajo.

Considerando la inmensa bibliografía que existe sobre el tema, encontraremos que también se define a un proceso, trabajo o tarea, de cualquiera de las formas siguientes:

- Un programa en ejecución.
- Una instancia de un programa ejecutado en un computador.
- La entidad que se puede asignar y ejecutar en un procesador.
- Una unidad de actividad que se caracteriza por la ejecución de una secuencia de instrucciones, un estado actual, y un conjunto de recursos del sistema asociados.
- Una entidad que consiste en un número de elementos. Los dos elementos esenciales serían el código de programa (que puede compartirse con otros procesos que estén ejecutando el mismo programa) y un conjunto de datos asociados a dicho código.
- Un programa que está alojado en la memoria RAM

Como se ha mencionado anteriormente, un proceso es un programa en ejecución. El estado de la actividad actual de un proceso está representado por el valor del contador de programa y el contenido de los registros del procesador. La disposición de la memoria de un proceso se suele dividir en varias secciones y se muestra en la figura 1.

Estas secciones incluyen:

- Sección de texto: el código ejecutable
- Sección de datos: variables globales

- Sección del cúmulo: memoria que se asigna dinámicamente durante la ejecución del programa
- Sección de la pila: almacenamiento temporal de datos al invocar funciones (como parámetros de funciones, direcciones de retorno y variables locales)

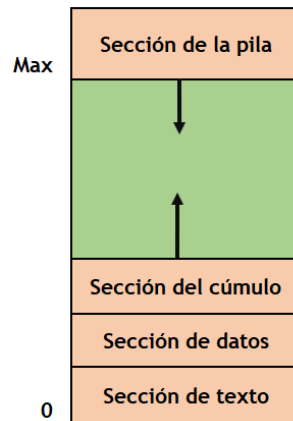


Figura 1

Obsérvese que los tamaños de las secciones de texto y datos son fijos, ya que sus tamaños no cambian durante la ejecución del programa. Sin embargo, las secciones de la pila y el cúmulo pueden reducirse y crecer dinámicamente durante la ejecución del programa. Cada vez que se llama a una función, se introduce en la pila un registro de activación que contiene los parámetros de la función, las variables locales y la dirección de retorno; cuando se devuelve el control de la función, el registro de activación se retira de la pila. Del mismo modo, el cúmulo crecerá a medida que la memoria se asigne dinámicamente, y se reducirá cuando la memoria se devuelva al sistema. Aunque las secciones de la pila y el cúmulo crecen la una hacia la otra, el sistema operativo debe asegurarse de que no se solapen.

Se debe destacar que un programa por sí mismo no es un proceso. Un programa es una entidad pasiva, como un archivo que contiene una lista de instrucciones almacenadas en el disco (a menudo llamado archivo ejecutable). En cambio, un proceso es una entidad activa, con un contador de programa que especifica la siguiente instrucción a ejecutar y un conjunto de recursos asociados. Un programa se convierte en un proceso cuando se carga un archivo ejecutable en la memoria. Dos técnicas comunes para cargar archivos ejecutables son hacer doble clic en un icono que representa el archivo ejecutable e introducir el nombre del archivo ejecutable en la línea de comandos (como en prog.exe o a.out).

Aunque dos procesos pueden estar asociados a un mismo programa, se consideran sin embargo dos secuencias de ejecución distintas. Por ejemplo, varios usuarios pueden estar ejecutando diferentes copias del programa de correo, o el mismo usuario puede invocar muchas copias del programa de navegación web. Cada uno de ellos es un proceso separado; y aunque las secciones de texto son equivalentes, las secciones de datos, el montón y la pila varían. También es común tener un proceso que engendra muchos procesos mientras se ejecuta.

Téngase en cuenta que un proceso puede ser en sí mismo un entorno de ejecución para otro código. El entorno de programación Java es un buen ejemplo. En la mayoría de las circunstancias, un programa Java ejecutable se ejecuta dentro de la máquina virtual Java (JVM). La JVM se ejecuta como un proceso que interpreta el código Java cargado y realiza

acciones (mediante instrucciones nativas de la máquina) en nombre de ese código. Por ejemplo, para ejecutar el programa Java compilado Program.class, introduciríamos: java Program

El comando java ejecuta la JVM como un proceso ordinario, que a su vez ejecuta el programa Java Program en la máquina virtual. El concepto es el mismo que el de simulación, salvo que el código, en lugar de estar escrito para un conjunto de instrucciones diferente, está escrito en el lenguaje Java.

Un proceso necesitará recursos, tales como CPU, memoria, archivos, etc., para llevar a cabo su tarea. Poseerá un espacio de direcciones protegido, actuará como una unidad de asignación de recursos y tendrá al menos una hebra mientras que el sistema operativo tendrá la responsabilidad de gestionar los siguientes aspectos del proceso:

- Su creación y destrucción.
- Su suspensión y reanudación.
- La asignación y mantenimiento de los recursos del proceso.
- Proporcionar mecanismos de sincronización y comunicación.
- Gestionar los interbloqueos.

Estados de un proceso. Cambios de estado

Como se acaba de comentar, para que un programa se ejecute, se crea un proceso, o tarea, para ese programa. Desde el punto de vista del procesador, éste ejecuta instrucciones de su repertorio en alguna secuencia dictada por los valores cambiantes del contador de programa. A lo largo del tiempo, el contador de programa puede referirse al código de diferentes programas que forman parte de diferentes procesos. Desde el punto de vista de un programa individual, su ejecución implica una secuencia de instrucciones dentro de ese programa.

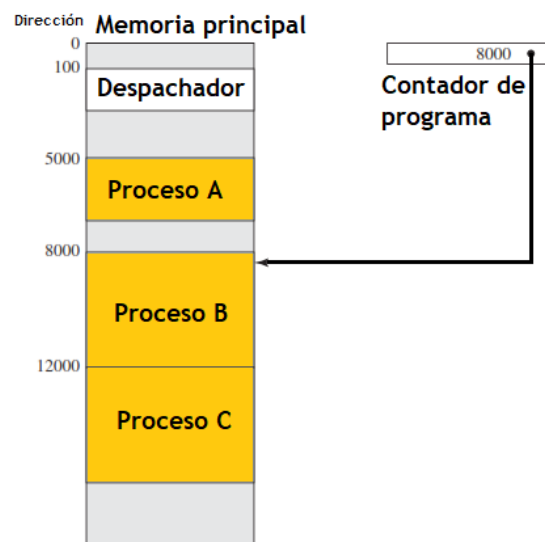


Figura 2

Podemos caracterizar el comportamiento de un proceso individual enumerando la secuencia de instrucciones que se ejecutan para ese proceso. Este listado se denomina *traza del proceso*. Podemos caracterizar el comportamiento del procesador mostrando cómo se intercalan las

trazas de los distintos procesos. Consideremos un ejemplo muy sencillo. La figura 2 muestra una disposición de memoria de tres procesos.

Para simplificar la discusión, suponemos que no se utiliza la memoria virtual; por lo tanto, los tres procesos están representados por programas que están completamente cargados en la memoria principal. Además, hay un pequeño programa llamado *despachador* que cambia el procesador de un proceso a otro. La figura 3 muestra los desarrollos de cada uno de los procesos durante la primera parte de su ejecución. Se muestran las primeras 12 instrucciones ejecutadas en los procesos A y C. El proceso B ejecuta cuatro instrucciones, y suponemos que la cuarta instrucción invoca una operación de E/S por la que el proceso debe esperar.

5000	8000	12000
5001	8001	12001
5002	8002	12002
5003	8003	12003
5004		12004
5005		12005
5006		12006
5007		12007
5008		12008
5009		12009
5010		12010
5011		12011
(a) Traza del Proceso A	(b) Traza del Proceso B	(c) Traza del Proceso C

Figura 3

Veamos ahora estas trazas desde el punto de vista del procesador. La figura 4 muestra las trazas intercaladas resultantes de los primeros 52 ciclos de instrucción (por comodidad, los ciclos de instrucción están numerados).

En esta figura, las áreas sombreadas representan el código ejecutado por el despachador; la primera y tercera columnas cuentan los ciclos de instrucción; la segunda y cuarta columnas muestran la dirección de la instrucción que se está ejecutando. La misma secuencia de instrucciones es ejecutada por el despachador en cada instancia porque la misma funcionalidad del despachador está siendo ejecutada. Suponemos que el sistema operativo sólo permite que un proceso continúe su ejecución durante un máximo de seis ciclos de instrucción, tras los cuales se interrumpe; esto evita que un solo proceso monopolice el tiempo del procesador. Como muestra la figura 4, se ejecutan las seis primeras instrucciones del proceso A, seguidas de un tiempo de espera y la ejecución de algún código en el despachador, que ejecuta seis instrucciones antes de pasar el control al proceso B (El pequeño número de instrucciones ejecutadas para los procesos y el despachador son irrealmente bajas; se utilizan en este ejemplo simplificado para aclarar la discusión.). Después de ejecutar cuatro instrucciones, el proceso B solicita una acción de E/S para la que debe esperar. Por lo tanto, el procesador deja de ejecutar el proceso B y pasa, a través del despachador, al proceso C. Después de un tiempo de espera, el procesador vuelve al proceso A. Cuando este proceso se agota, el proceso B sigue

esperando a que se complete la operación de E/S, por lo que el despachador pasa de nuevo al proceso C.

1	5000	27	12004
2	5001	28	12005
3	5002	----- Tiempo de espera	
4	5003	29	100
5	5004	30	101
6	5005	31	102
----- Tiempo de espera		32	103
7	100	33	104
8	101	34	105
9	102	35	5006
10	103	36	5007
11	104	37	5008
12	105	38	5009
13	8000	39	5010
14	8001	40	5011
15	8002	----- Tiempo de espera	
16	8003	41	100
----- Solicitud de E/S		42	101
17	100	43	102
18	101	44	103
19	102	45	104
20	103	46	105
21	104	47	12006
22	105	48	12007
23	12000	49	12008
24	12001	50	12009
25	12002	51	12010
26	12003	52	12011
		----- Tiempo de espera	

Figura 4

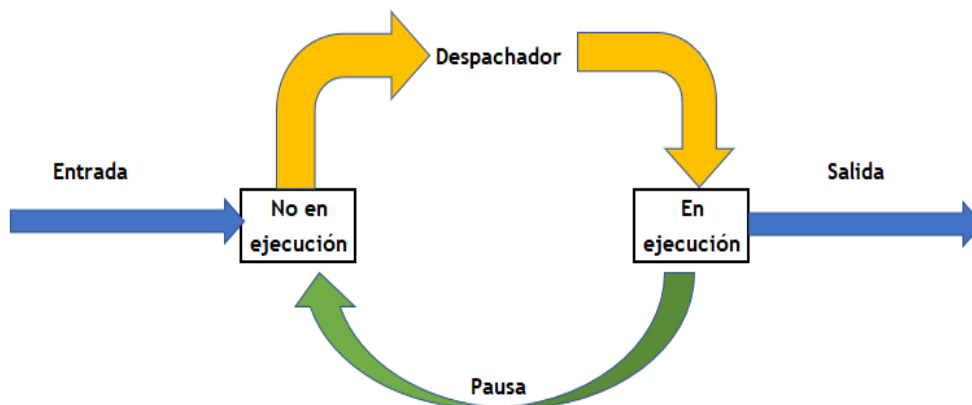
Un modelo de procesos de dos estados

La principal responsabilidad del sistema operativo es controlar la ejecución de los procesos; esto incluye determinar el patrón de intercalación para la ejecución y asignar recursos a los procesos. El primer paso en el diseño de un sistema operativo para controlar los procesos es describir el comportamiento que nos gustaría que mostraran los procesos.

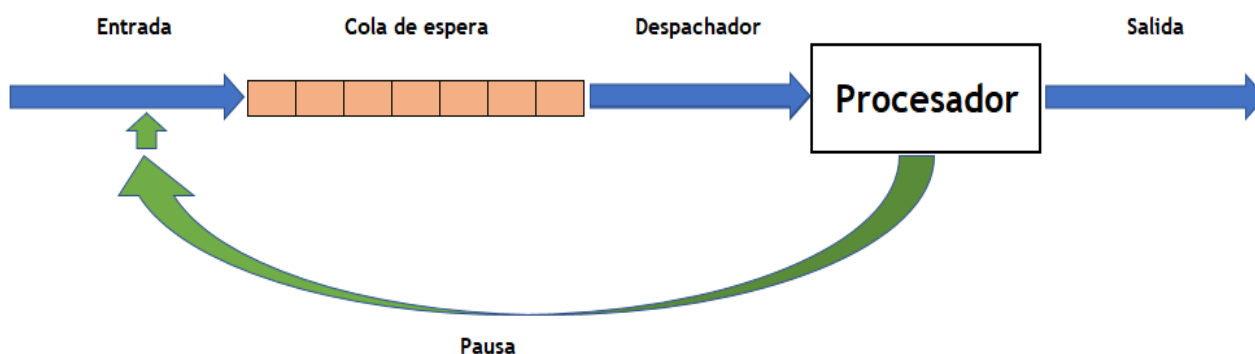
Podemos construir el modelo más simple posible observando que, en cualquier momento, un proceso está siendo ejecutado por un procesador o no. En este modelo, un proceso puede estar en uno de dos estados: *En ejecución* o *No en ejecución*, como se muestra en la figura 5(a). Cuando el sistema operativo crea un nuevo proceso, crea un bloque de control de proceso para el proceso e introduce ese proceso en el sistema en el estado de No Ejecutado. El proceso existe, es conocido por el sistema operativo, y está esperando una oportunidad para ejecutarse. De vez en cuando, el proceso que se está ejecutando actualmente será interrumpido y la parte

del despachador del sistema operativo seleccionará algún otro proceso para ejecutarlo. El proceso anterior pasa del estado de ejecución al estado de no ejecución, y uno de los otros procesos pasa al estado de ejecución.

A partir de este sencillo modelo, ya podemos empezar a apreciar algunos de los elementos de diseño del sistema operativo. Cada proceso debe ser representado de alguna manera para que el sistema operativo pueda seguirlo. Es decir, debe haber alguna información relacionada con cada proceso, incluyendo el estado actual y la ubicación en la memoria; este es el bloque de control del proceso.



(a) Diagrama de transición de estados



(b) Diagrama de colas

Figura 5

Los procesos que no se están ejecutando deben mantenerse en algún tipo de cola, esperando su turno para ejecutarse. La figura 5(b) sugiere una estructura. Hay una cola única en la que cada entrada es un puntero al bloque de control de procesos de un proceso concreto. Alternativamente, la cola puede consistir en una lista enlazada de bloques de datos, en la que cada bloque representa un proceso; exploraremos esta última implementación posteriormente.

Podemos describir el comportamiento del despachador en términos de este diagrama de colas. Un proceso que se interrumpe se transfiere a la cola de procesos en espera. Alternativamente, si el proceso se ha completado o abortado, se descarta (sale del sistema). En cualquier caso, el despachador toma otro proceso de la cola para ejecutarlo.

Creación y terminación de procesos

Antes de refinar nuestro modelo simple de dos estados, será útil discutir la creación y terminación de los procesos; en última instancia, e independientemente del modelo de comportamiento de los procesos que se utilice, la vida de un proceso está limitada por su creación y terminación.

Creación de procesos.

Cuando se va a añadir un nuevo proceso a los que se están gestionando actualmente, el sistema operativo construye las estructuras de datos que se utilizan para gestionar el proceso y asigna un espacio de direcciones en la memoria principal al proceso. Describimos estas estructuras más adelante. Estas acciones constituyen la creación de un nuevo proceso.

Cuatro eventos comunes conducen a la creación de un proceso, como se indica en la tabla *Razones para la creación de procesos*. En un entorno por lotes, se crea un proceso en respuesta al envío de un trabajo. En un entorno interactivo, se crea un proceso cuando un nuevo usuario intenta conectarse. En ambos casos, el sistema operativo es responsable de la creación del nuevo proceso. Un sistema operativo también puede crear un proceso en nombre de una aplicación. Por ejemplo, si un usuario solicita que se imprima un archivo, el sistema operativo puede crear un proceso que gestione la impresión. El proceso solicitante puede así proceder independientemente del tiempo requerido para completar la tarea de impresión.

Razones para la creación de procesos	
Nuevo trabajo por lotes	El sistema operativo dispone de una secuencia de control de trabajos por lotes, normalmente en cinta o disco. Cuando el sistema operativo está preparado para asumir un nuevo trabajo, leerá la siguiente secuencia de comandos de control de trabajo.
Inicio de sesión interactivo	Un usuario en un terminal se conecta al sistema.
Creada por sistema operativo para prestar un servicio	El sistema operativo puede crear un proceso para realizar una función en nombre de un programa de usuario, sin que éste tenga que esperar (por ejemplo, un proceso para controlar la impresión).
Generado por el proceso existente	Con fines de modularidad o para explotar el paralelismo, un programa de usuario puede dictar la creación de una serie de procesos.

Tradicionalmente, el sistema operativo creaba todos los procesos de forma transparente para el usuario o el programa de aplicación, y esto sigue siendo habitual en muchos sistemas operativos contemporáneos. Sin embargo, puede ser útil permitir que un proceso provoque la creación de otro. Por ejemplo, un proceso de aplicación puede generar otro proceso para recibir los datos que la aplicación está generando y para organizar esos datos en una forma adecuada para su posterior análisis. El nuevo proceso se ejecuta en paralelo al proceso original y se activa cada cierto tiempo cuando hay nuevos datos disponibles. Esta disposición puede ser muy útil para estructurar la aplicación. Como otro ejemplo, un proceso de servidor (por ejemplo, servidor de impresión, servidor de archivos) puede generar un nuevo proceso para cada solicitud que

maneja. Cuando el sistema operativo crea un proceso a petición explícita de otro proceso, la acción se denomina generación de procesos. Cuando un proceso engendra a otro, el primero se denomina *proceso padre*, y el proceso engendrado se denomina *proceso hijo*. Normalmente, los procesos relacionados deben comunicarse y cooperar entre sí, pero lograr esta cooperación es una tarea difícil para el programador.

Terminación del proceso.

La tabla *Motivos de la finalización del proceso* resume las razones típicas para la terminación de un proceso. Cualquier sistema informático debe proporcionar un medio para que un proceso indique su finalización. Un trabajo por lotes debe incluir una instrucción *Halt* o una llamada de servicio explícita del sistema operativo para la terminación. En el primer caso, la instrucción *Halt* generará una interrupción para alertar al sistema operativo de que un proceso ha finalizado. En el caso de una aplicación interactiva, la acción del usuario indicará cuándo se ha completado el proceso. Por ejemplo, en un sistema de tiempo compartido, el proceso para un usuario particular debe terminar cuando el usuario se desconecta o apaga su terminal. En un ordenador personal o estación de trabajo, un usuario puede salir de una aplicación (por ejemplo, un procesador de textos o una hoja de cálculo). Todas estas acciones acaban provocando una petición de servicio al sistema operativo para que termine el proceso solicitante. Además, varias condiciones de error y fallo pueden llevar a la terminación de un proceso. La tabla *Motivos de la finalización del proceso* enumera algunas de las condiciones más comúnmente reconocidas. Un sistema operativo indulgente podría, en algunos casos, permitir al usuario recuperarse de un fallo sin terminar el proceso. Por ejemplo, si un usuario solicita acceso a un archivo y ese acceso es denegado, el sistema operativo podría simplemente informar al usuario de que el acceso está denegado y permitir que el proceso continúe.

Motivos de la finalización del proceso	
Finalización normal	El proceso ejecuta una llamada de servicio del sistema operativo para indicar que ha terminado de ejecutarse.
Se ha excedido el límite de tiempo	El proceso se ha ejecutado durante más tiempo que el límite de tiempo total especificado. Hay varias posibilidades para el tipo de tiempo que se mide. Estas incluyen el tiempo total transcurrido (tiempo de reloj), la cantidad de tiempo de ejecución y, en el caso de un proceso interactivo, la cantidad de tiempo desde que el usuario proporcionó cualquier entrada por última vez.
Memoria no disponible	El proceso requiere más memoria de la que el sistema puede proporcionar.
Violación de los límites	El proceso intenta acceder a una posición de memoria a la que no está permitido acceder.
Error de protección	El proceso intenta utilizar un recurso, como un archivo, que no está autorizado a utilizar, o intenta utilizarlo de forma inadecuada, como escribir en un archivo de sólo lectura.

Motivos de la finalización del proceso	
Error aritmético	El proceso intenta un cálculo prohibido, como la división por cero, o intenta almacenar números más grandes de lo que el hardware puede acomodar.
Tiempo excedido	El proceso ha esperado más de un máximo especificado para que se produzca un determinado evento.
Fallo de E/S	Se produce un error durante la entrada o la salida, como la incapacidad de encontrar un archivo, la imposibilidad de leer o escribir después de un número máximo especificado de intentos (cuando, por ejemplo, se encuentra un área defectuosa en una cinta), o una operación no válida (como la lectura de la impresora de línea).
Instrucción no válida	El proceso intenta ejecutar una instrucción inexistente (a menudo el resultado de bifurcarse en un área de datos e intentar ejecutar los datos).
Instrucción privilegiada	El proceso intenta utilizar una instrucción reservada para el sistema operativo.
Uso indebido de datos	Un dato es de tipo incorrecto o no está inicializado.
Intervención del operador o del sistema operativo	Por alguna razón, el operador o el sistema operativo ha terminado el proceso (por ejemplo, si existe un bloqueo).
Terminación del padre	Cuando un padre termina, el sistema operativo puede terminar automáticamente todos los hijos de ese padre.
Solicitud del padre	Un proceso padre suele tener la autoridad para acabar con cualquiera de sus descendientes.

Finalmente, en algunos sistemas operativos, un proceso puede ser terminado por el proceso que lo creó o cuando el proceso padre es terminado por sí mismo.

Un modelo de cinco estados

Si todos los procesos estuvieran siempre listos para ejecutarse, entonces la disciplina de colas sugerida por la figura 5b sería efectiva. La cola es una lista de "primero en entrar, primero en salir", y el procesador opera en forma de round-robin sobre los procesos disponibles (a cada proceso de la cola se le da una cierta cantidad de tiempo, a su vez, para ejecutar y luego se devuelve a la cola, a menos que se bloquee). Sin embargo, incluso con el sencillo ejemplo que hemos descrito, esta implementación es inadecuada: Algunos procesos en el estado de No ejecutado están listos para ejecutarse, mientras que otros están bloqueados, esperando que se complete una operación de E/S. Por lo tanto, utilizando una sola cola, el despachador no podría simplemente seleccionar el proceso en el extremo más antiguo de la cola. Más bien, el despachador tendría que escanear la lista buscando el proceso que no está bloqueado y que ha estado en la cola por más tiempo.

Una forma más natural de manejar esta situación es dividir el estado de no ejecución en dos estados: Listo y Bloqueado. Esto se muestra en la figura 6. Por si fuera poco, hemos añadido dos estados adicionales que resultarán útiles. Los cinco estados de este nuevo diagrama son:

- **En ejecución:** El proceso que se está ejecutando actualmente. Para este módulo, asumiremos un computador con un solo procesador, por lo que como máximo un proceso a la vez puede estar en este estado.
- **Listo:** Un proceso que está preparado para ejecutarse cuando se le dé la oportunidad.
- **Bloqueado/En espera:** *En espera* es un término alternativo frecuentemente usado para Bloqueado como estado de proceso. Generalmente, usaremos Bloqueado, pero los términos son intercambiables. Un proceso que no puede ejecutarse hasta que ocurra algún evento, como la finalización de una operación de E/S.
- **Nuevo:** Un proceso que acaba de ser creado pero que aún no ha sido admitido por el sistema operativo en el conjunto de procesos ejecutables. Normalmente, un proceso nuevo no se ha cargado todavía en la memoria principal, aunque se ha creado su bloque de control de procesos.
- **Terminado:** Un proceso que ha sido liberado del conjunto de procesos ejecutables por el sistema operativo, ya sea porque se ha detenido o porque ha abortado por alguna razón.



Figura 6

Los estados Nuevo y Terminado son construcciones útiles para la gestión de procesos. El estado Nuevo corresponde a un proceso que acaba de ser definido. Por ejemplo, si un nuevo usuario intenta conectarse a un sistema de tiempo compartido o se envía un nuevo trabajo por lotes para su ejecución, el sistema operativo puede definir un nuevo proceso en dos etapas. En primer lugar, el sistema operativo realiza las tareas de mantenimiento necesarias. Se asocia un identificador al proceso. Se asignan y construyen las tablas que se necesitarán para gestionar el proceso. En este punto, el proceso está en el estado Nuevo. Esto significa que el sistema operativo ha realizado las acciones necesarias para crear el proceso, pero no se ha comprometido a la ejecución de este. Por ejemplo, el sistema operativo puede limitar el número de procesos que puede haber en el sistema por razones de rendimiento o de limitación de la memoria principal. Mientras un proceso está en el estado Nuevo, la información relativa al proceso que necesita el sistema operativo se mantiene en tablas de control en la memoria principal. Sin embargo, el proceso en sí no está en la memoria principal. Es decir, el código del programa a ejecutar no está en la memoria principal, y no se ha asignado espacio para los datos asociados a ese programa. Mientras el proceso está en el estado Nuevo, el programa permanece en el almacenamiento secundario, típicamente el almacenamiento en disco. En la discusión de

este párrafo, ignoramos el concepto de memoria virtual. En los sistemas que soportan la memoria virtual, cuando un proceso pasa de Nuevo a Listo, el código y los datos de su programa se cargan en la memoria virtual.

Del mismo modo, un proceso sale de un sistema en dos etapas. En primer lugar, un proceso se termina cuando alcanza un punto de finalización natural, cuando aborta debido a un error irrecuperable, o cuando otro proceso con la autoridad apropiada hace que el proceso aborte. La terminación hace que el proceso pase al estado Terminado. En este punto, el proceso ya no puede ser ejecutado. Las tablas y otra información asociada al trabajo son conservadas temporalmente por el sistema operativo, lo que da tiempo a que los programas auxiliares o de apoyo extraigan cualquier información necesaria. Por ejemplo, un programa de contabilidad puede necesitar registrar el tiempo del procesador y otros recursos utilizados por el proceso para fines de facturación. Un programa de utilidad puede necesitar extraer información sobre el historial del proceso para propósitos relacionados con el rendimiento o el análisis de utilización. Una vez que estos programas han extraído la información necesaria, el sistema operativo ya no necesita mantener ningún dato relacionado con el proceso y éste se elimina del sistema.

La figura 6 indica los tipos de eventos que conducen a cada transición de estado para un proceso; las transiciones posibles son las siguientes:

- **Nulo >> Nuevo:** Se crea un nuevo proceso para ejecutar un programa. Este evento se produce por cualquiera de las razones enumeradas en la tabla *Razones para la creación de procesos*.
- **Nuevo >> Listo:** El sistema operativo moverá un proceso del estado Nuevo al estado Listo cuando esté preparado para asumir un proceso adicional. La mayoría de los sistemas establecen algún límite basado en el número de procesos existentes o la cantidad de memoria virtual comprometida con los procesos existentes. Este límite asegura que no hay tantos procesos activos como para degradar el rendimiento.
- **Listo >> En ejecución:** Cuando llega el momento de seleccionar un proceso para ejecutarlo, el sistema operativo elige uno de los procesos en estado Listo. Este es el trabajo del programador o despachador. La programación se explora en la cuarta parte.
- **En ejecución >> Terminado:** El proceso que se está ejecutando actualmente es terminado por el sistema operativo si el proceso indica que ha terminado, o si aborta. Véase la tabla *Motivos de la finalización del proceso*.
- **En ejecución >> Listo:** La razón más común para esta transición es que el proceso en ejecución ha alcanzado el tiempo máximo permitido para la ejecución ininterrumpida; prácticamente todos los sistemas operativos de multiprogramación imponen este tipo de disciplina de tiempo. Existen otras causas alternativas para esta transición, que no están implementadas en todos los sistemas operativos. Es especialmente importante el caso en el que el sistema operativo asigna diferentes niveles de prioridad a los distintos procesos. Supongamos, por ejemplo, que el proceso A se está ejecutando con un nivel de prioridad determinado, y el proceso B, con un nivel de prioridad superior, está bloqueado. Si el sistema operativo se entera de que el evento por el que el proceso B ha estado esperando ha ocurrido, moviendo a B a un estado listo, entonces puede interrumpir el proceso A y despachar el proceso B. Decimos que el sistema operativo se ha adelantado al proceso A.

Finalmente, un proceso puede liberar voluntariamente el control del procesador. Un ejemplo es un proceso en segundo plano que realiza alguna función de contabilidad o mantenimiento periódicamente.

- **En ejecución >> Bloqueado:** Un proceso se pone en el estado de Bloqueo si solicita algo por lo que debe esperar. Una petición al sistema operativo suele ser en forma de llamada a un servicio del sistema; es decir, una llamada del programa en ejecución a un procedimiento que forma parte del código del sistema operativo. Por ejemplo, un proceso puede solicitar un servicio al sistema operativo que éste no está preparado para realizar inmediatamente. Puede solicitar un recurso, como un archivo o una sección compartida de la memoria virtual, que no está disponible inmediatamente. O el proceso puede iniciar una acción, como una operación de E/S, que debe completarse antes de que el proceso pueda continuar. Cuando los procesos se comunican entre sí, un proceso puede estar bloqueado cuando está esperando que otro proceso proporcione datos o esperando un mensaje de otro proceso.
- **Bloqueado >> Listo:** Un proceso en estado de Bloqueo pasa al estado de Listo cuando se produce el evento por el que ha estado esperando.
- **Listo >> Terminado:** Por claridad, esta transición no se muestra en el diagrama de estados. En algunos sistemas, un padre puede terminar un proceso hijo en cualquier momento. Además, si un padre termina, todos los procesos hijos asociados a ese padre pueden ser terminados.
- **Bloqueado >> Terminado:** Se aplican los comentarios del punto anterior.

Volviendo a nuestro sencillo ejemplo, la figura 7 muestra la transición de cada proceso entre los estados.

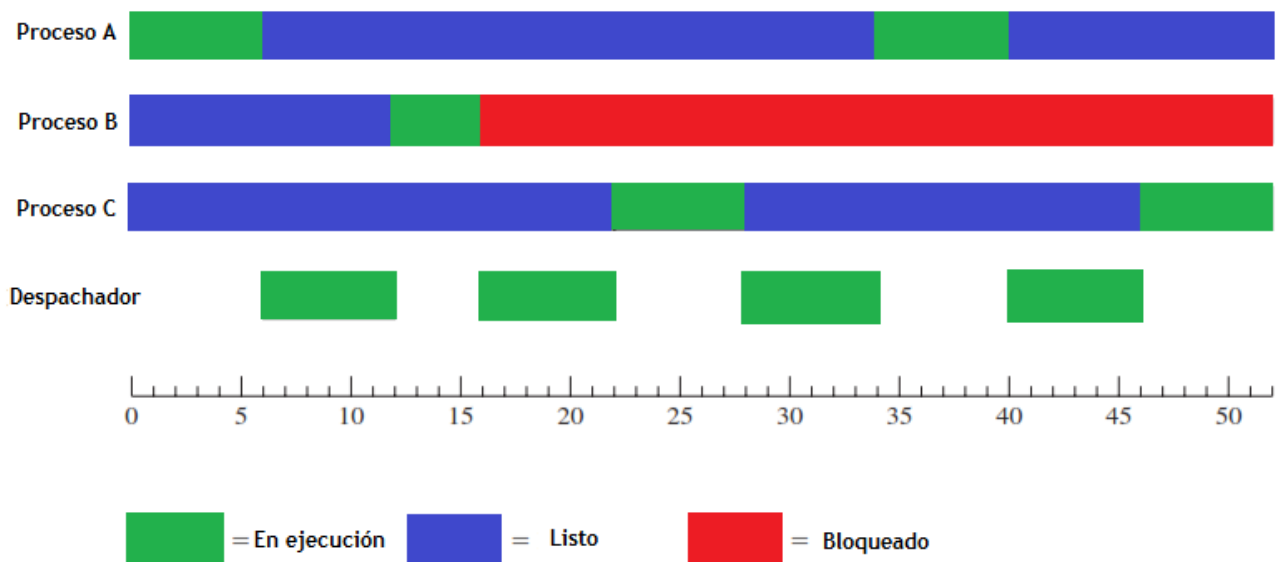


Figura 7

La figura 8(a) sugiere la forma en que podría implementarse una disciplina de colas con dos colas: una cola de Listos y otra de Bloqueados. A medida que cada proceso es admitido en el sistema, se coloca en la cola de Listos. Cuando llega el momento de que el sistema operativo elija otro proceso para ejecutar, selecciona uno de la cola de Listos. En ausencia de un

esquema de prioridades, esto puede ser una simple cola de primero en entrar, primero en salir. Cuando un proceso en ejecución se retira de la ejecución, se termina o se coloca en la cola de Listos o Bloqueados, dependiendo de las circunstancias. Por último, cuando se produce un evento, cualquier proceso en la cola de bloqueados que haya estado esperando sólo ese evento se mueve a la cola de listos.

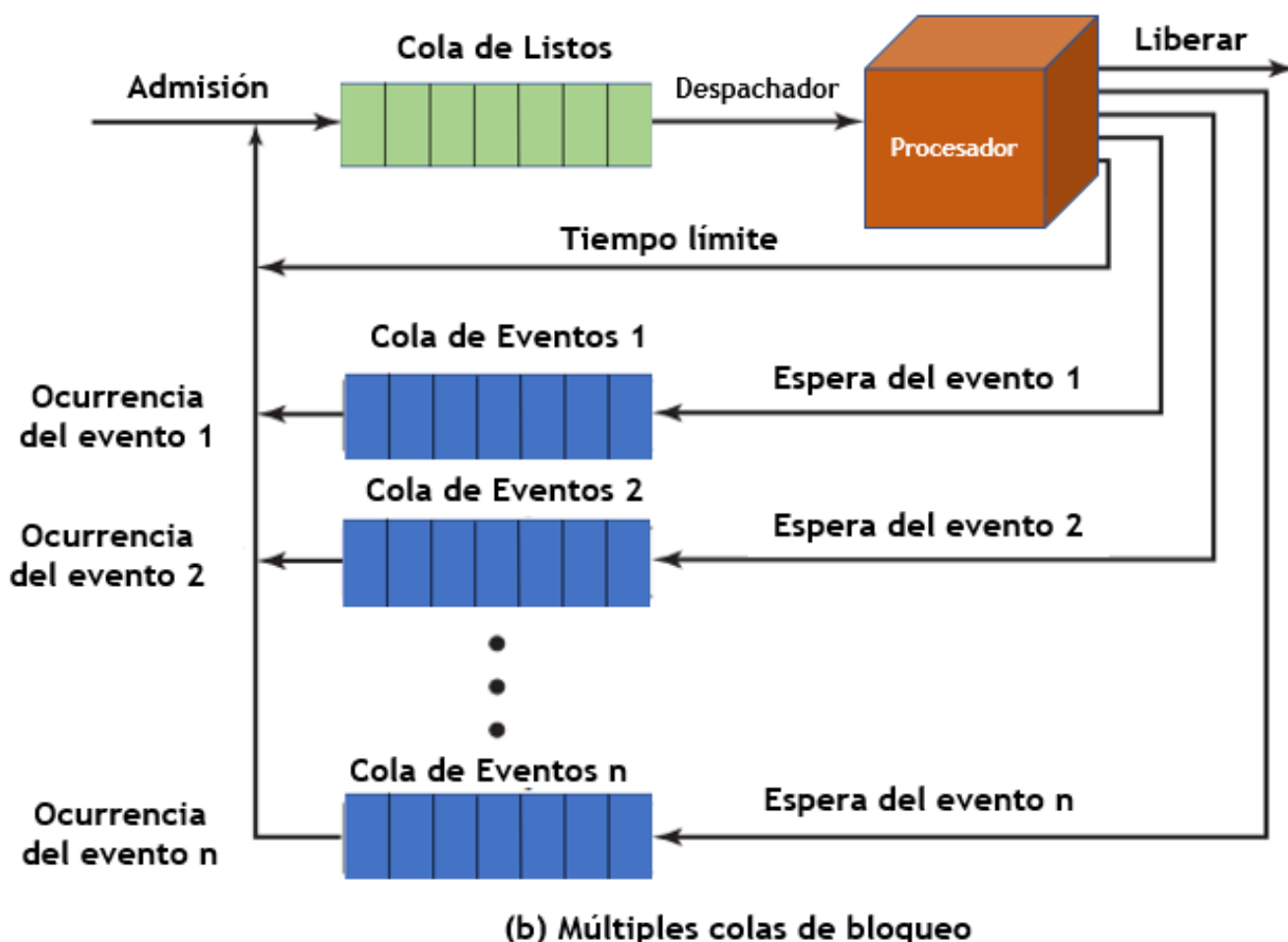
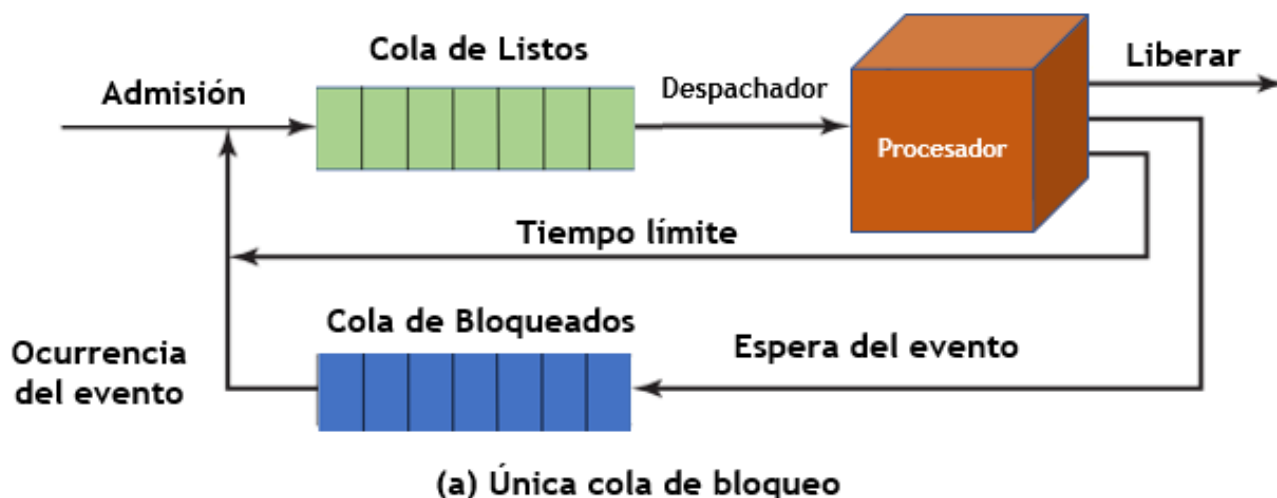


Figura 8

Esta última disposición significa que, cuando se produce un evento, el sistema operativo debe escanear toda la cola de bloqueados, buscando aquellos procesos que estén esperando ese

© Universidad de Palermo Prohibida la reproducción total o parcial de imágenes y textos

evento. En un sistema operativo grande, podría haber cientos o incluso miles de procesos en esa cola. Por lo tanto, sería más eficiente tener varias colas, una para cada evento. Entonces, cuando se produzca el evento, toda la lista de procesos de la cola correspondiente puede pasar al estado Listo (Figura 8(b)).

Un último refinamiento: Si el envío de procesos está dictado por un esquema de prioridades, entonces sería conveniente tener un número de colas de Listo, una para cada nivel de prioridad. El sistema operativo podría entonces determinar fácilmente cuál es el proceso listo de mayor prioridad que ha estado esperando más tiempo.

Procesos suspendidos y la necesidad de intercambiar.

Los tres principales estados que acabamos de describir (Listo, En ejecución, Bloqueado) proporcionan una forma sistemática de modelar el comportamiento de los procesos y guían la implementación del sistema operativo. Algunos sistemas operativos se construyen utilizando sólo estos tres estados. Sin embargo, hay una buena justificación para añadir otros estados al modelo. Para ver el beneficio de estos nuevos estados, considere un sistema que no emplea memoria virtual. Cada proceso que se ejecute debe cargarse completamente en la memoria principal. Así, en la figura 8(b), todos los procesos de todas las colas deben residir en la memoria principal.

Recordemos que la razón de toda esta elaborada maquinaria es que las actividades de E/S son mucho más lentas que la computadora y, por tanto, el procesador en un sistema de monoprogramación está inactivo la mayor parte del tiempo. Pero la disposición de la figura 8(b) no resuelve del todo el problema. Es cierto que, en este caso, la memoria alberga múltiples procesos y que el procesador puede pasar a otro proceso cuando uno de ellos se bloquea. Pero el procesador es mucho más rápido que la E/S por lo que será muy común que todos los procesos en memoria estén esperando la E/S. Así, incluso con multiprogramación, un procesador podría estar inactivo la mayor parte del tiempo.

¿Qué hacer? Se podría ampliar la memoria principal para dar cabida a más procesos. Pero este enfoque tiene dos defectos. En primer lugar, hay un costo asociado a la memoria principal que, aunque es pequeño por cada byte, empieza a sumarse a medida que nos acercamos a los gigabytes de almacenamiento. En segundo lugar, el apetito de los programas por la memoria ha crecido tan rápido como el costo de esta ha disminuido. Por tanto, una mayor memoria da lugar a procesos más grandes, no a más procesos.

Otra solución es el swapping (intercambio), que consiste en trasladar parte o la totalidad de un proceso de la memoria principal al disco. Cuando ninguno de los procesos de la memoria principal está en el estado Listo, el sistema operativo intercambia uno de los procesos bloqueados en el disco en una cola de suspensión. Esta es una cola de procesos existentes que han sido temporalmente expulsados de la memoria principal o suspendidos. El sistema operativo entonces trae otro proceso de la cola de suspensión, o satisface una solicitud de nuevo proceso. La ejecución continúa entonces con el proceso recién llegado.

El intercambio, sin embargo, es una operación de E/S, y por lo tanto existe la posibilidad de empeorar el problema, no de mejorarlo. Pero debido a que la E/S del disco es generalmente la

E/S más rápida en un sistema (por ejemplo, comparada con la E/S de la cinta o de la impresora), el intercambio generalmente mejorará el rendimiento.

Con el uso de swapping como se acaba de describir, hay que añadir otro estado a nuestro modelo de comportamiento de procesos (Figura 9(a)): el estado Suspendido. Cuando todos los procesos en la memoria principal están en el estado Bloqueado, el sistema operativo puede suspender un proceso poniéndolo en el estado Suspendido y transfiriéndolo al disco. El espacio que se libera en la memoria principal puede entonces utilizarse para traer otro proceso. Cuando el sistema operativo ha realizado una operación de Swapping-Out, tiene dos opciones para seleccionar un proceso para traer a la memoria principal:

- Puede admitir un proceso recién creado, o...
- Puede traer un proceso previamente suspendido.

Parece que la preferencia debería ser traer un proceso previamente suspendido, para darle servicio en lugar de aumentar la carga total del sistema. Pero esta línea de razonamiento presenta una dificultad. Todos los procesos que han sido suspendidos estaban en el estado Bloqueado en el momento de la suspensión. Evidentemente, no serviría de nada devolver un proceso bloqueado a la memoria principal, porque todavía no está listo para su ejecución. Reconozcamos, sin embargo, que cada proceso en el estado Suspendido estaba originalmente bloqueado por un evento particular. Cuando ese evento ocurre, el proceso no está bloqueado y está potencialmente disponible para su ejecución.

Por lo tanto, tenemos que repensar este aspecto del diseño. Aquí hay dos conceptos independientes: si un proceso está esperando un evento (bloqueado o no) y si un proceso ha sido intercambiado fuera de la memoria principal (suspendido o no). Para acomodar esta combinación de 2 x 2, necesitamos cuatro estados:

- **Listo:** El proceso está en la memoria principal y disponible para su ejecución.
- **Bloqueado:** El proceso está en la memoria principal y a la espera de un evento.
- **Bloqueado/Suspendido:** El proceso se encuentra en la memoria secundaria y está esperando un evento.
- **Listo/Suspendido:** El proceso está en la memoria secundaria, pero está disponible para su ejecución en cuanto se cargue en la memoria principal.

Antes de ver un diagrama de transición de estados que abarque los dos nuevos estados de suspensión, hay que mencionar otro punto. La discusión hasta ahora ha asumido que la memoria virtual no está en uso y que un proceso está todo en la memoria principal o todo fuera de la memoria principal. Con un esquema de memoria virtual, es posible ejecutar un proceso que esté sólo parcialmente en la memoria principal. Si se hace referencia a la dirección de un proceso que no está en la memoria principal, entonces se puede traer la porción apropiada del proceso. El uso de la memoria virtual parecería eliminar la necesidad de un intercambio explícito, porque cualquier dirección deseada en cualquier proceso deseado puede ser movida dentro o fuera de la memoria principal por el hardware de gestión de memoria del procesador. Sin embargo, el rendimiento de un sistema de memoria virtual puede colapsar si hay un número suficientemente grande de procesos activos, todos los cuales están parcialmente en la memoria

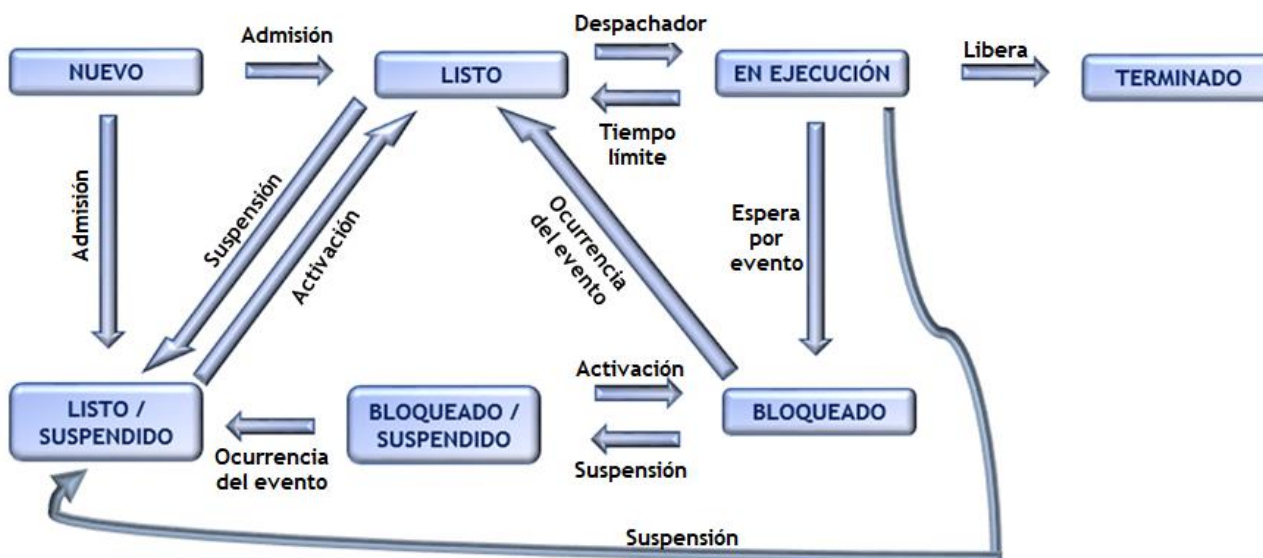
principal. Por lo tanto, incluso en un sistema de memoria virtual, el sistema operativo tendrá que intercambiar procesos explícita y completamente de vez en cuando en aras del rendimiento.

Veamos ahora, en la figura 9(b), el modelo de transición de estados que hemos desarrollado con una aclaración previa. En esa figura, las líneas de conexión entre los estados Nuevo y Listo en un caso, y los estados En ejecución y Listo/Suspendido en otro, indican transiciones posibles, pero no necesarias. Dicho esto, las nuevas transiciones importantes son las siguientes:

- **Bloqueado >> Bloqueado/Suspendido:** Si no hay procesos listos, entonces al menos un proceso bloqueado se intercambia para dejar espacio a otro proceso que no esté bloqueado. Esta transición puede hacerse incluso si hay procesos listos disponibles si el sistema operativo determina que el proceso que se está ejecutando actualmente o un proceso listo que le gustaría enviar requiere más memoria principal para mantener un rendimiento adecuado.



(a) Con un estado de suspensión



(b) Con dos estados de suspensión

Figura 9

- **Bloqueado/Suspendido >> Listo/Suspendido:** Un proceso en el estado de Bloqueo/Suspensión es movido al estado de Listo/Suspensión cuando ocurre el evento por el cual ha estado esperando. Tengamos en cuenta que esto requiere que la información de estado relativa a los procesos suspendidos sea accesible para el sistema operativo.
- **Listo/Suspendido >> Listo:** Cuando no hay procesos listos en la memoria principal, el sistema operativo necesitará traer uno para continuar la ejecución. Además, puede darse el caso de que un proceso en estado Listo/Suspendido tenga mayor prioridad que cualquiera de los procesos en estado Listo. En ese caso, el diseñador del sistema operativo puede dictar que es más importante llegar al proceso de mayor prioridad que minimizar el intercambio.
- **Listo >> Listo/Suspendido:** Normalmente, el sistema operativo prefiere suspender un proceso bloqueado en lugar de uno listo, porque el proceso listo puede ser ejecutado, mientras que el proceso bloqueado está ocupando espacio en la memoria principal y no puede ser ejecutado. Sin embargo, puede ser necesario suspender un proceso listo si es la única manera de liberar un bloque de memoria principal suficientemente grande. Además, el sistema operativo puede optar por suspender un proceso listo de menor prioridad en lugar de un proceso bloqueado de mayor prioridad si cree que el proceso bloqueado estará listo pronto.

Otras transiciones que vale la pena considerar son las siguientes:

- **Nuevo >> Listo/Suspendido y Nuevo >> Listo:** Cuando se crea un nuevo proceso, puede añadirse a la cola de Listos o a la cola de Listos/Suspendidos. En cualquiera de los dos casos, el sistema operativo debe crear un bloque de control de proceso y asignar un espacio de direcciones al proceso. Puede ser preferible que el sistema operativo realice estas tareas de mantenimiento en un momento temprano, para que pueda mantener un gran grupo de procesos que no estén bloqueados. Con esta estrategia, a menudo no habría suficiente espacio en la memoria principal para un nuevo proceso; de ahí el uso de la transición (Nuevo >> Listo/Suspendido). Por otro lado, podríamos argumentar que una filosofía *justo a tiempo* de crear procesos lo más tarde posible reduce la sobrecarga del sistema operativo y permite que éste realice las tareas de creación de procesos en un momento en el que el sistema está atascado con procesos bloqueados de todas formas.
- **Bloqueado/Suspendido >> Bloqueado:** La inclusión de esta transición puede parecer un diseño pobre. Después de todo, si un proceso no está listo para ejecutarse y no está ya en la memoria principal, ¿qué sentido tiene traerlo? Pero considere el siguiente escenario: Un proceso termina, liberando algo de memoria principal. Hay un proceso en la cola (Bloqueado/Suspendido) con una prioridad más alta que cualquiera de los procesos en la cola (Listo/Suspendido) y el sistema operativo tiene razones para creer que el evento de bloqueo para ese proceso ocurrirá pronto. En estas circunstancias, parece razonable traer un proceso bloqueado a la memoria principal con preferencia a un proceso listo.
- **Ejecución >> Listo/Suspendido:** Normalmente, un proceso en ejecución es movido al estado Listo cuando su asignación de tiempo expira. Sin embargo, si el sistema operativo está reemprendiendo el proceso porque un proceso de mayor prioridad en la cola de Bloqueados/Suspendidos acaba de ser desbloqueado, el sistema operativo podría mover el

proceso en ejecución directamente a la cola de Listos/Suspendidos y liberar algo de memoria principal.

- **Cualquier Estado >> Terminado:** Típicamente, un proceso termina mientras se está ejecutando, ya sea porque ha terminado o por alguna condición de fallo fatal. Sin embargo, en algunos sistemas operativos, un proceso puede ser terminado por el proceso que lo creó o cuando el propio proceso padre es terminado. Si esto se permite, entonces un proceso en cualquier estado puede ser movido al estado de Terminado.

Otras aplicaciones de la suspensión

Hasta ahora, hemos equiparado el concepto de proceso suspendido con el de un proceso que no está en la memoria principal. Un proceso que no está en la memoria principal no está disponible inmediatamente para su ejecución, esté o no esperando un evento. Podemos generalizar el concepto de proceso suspendido. Definamos un proceso suspendido como el que tiene las siguientes características:

- El proceso no está disponible inmediatamente para su ejecución.
- El proceso puede o no estar esperando un evento. Si lo está, esta condición de bloqueo es independiente de la condición de suspensión, y la ocurrencia del evento de bloqueo no permite que el proceso se ejecute inmediatamente.
- El proceso fue colocado en un estado de suspensión por un agente: ya sea él mismo, un proceso padre, o el sistema operativo, con el propósito de prevenir su ejecución.
- El proceso no puede ser removido de este estado hasta que el agente ordene explícitamente la remoción.

La tabla *Motivos de la suspensión de un proceso* enumera algunas razones para la suspensión de un proceso. Una de las razones que hemos discutido es proporcionar espacio de memoria, ya sea para traer un proceso Listo/Suspendido o para aumentar la memoria asignada a otros procesos Listos. El sistema operativo puede tener otras motivaciones para suspender un proceso. Por ejemplo, se puede emplear un proceso de auditoría o rastreo para monitorear la actividad en el sistema; el proceso puede ser usado para registrar el nivel de utilización de varios recursos (procesador, memoria, canales) y el ritmo de progreso de los procesos de usuario en el sistema. El sistema operativo, bajo el control del operador, puede activar y desactivar este proceso de vez en cuando. Si el sistema operativo detecta o sospecha un problema, puede suspender un proceso. Un ejemplo de esto es el interbloqueo. Como otro ejemplo, se detecta un problema en una línea de comunicaciones, y el operador hace que el sistema operativo suspenda el proceso que está usando la línea mientras se ejecutan algunas pruebas.

Motivos de la suspensión de un proceso	
Intercambio	El sistema operativo necesita liberar suficiente memoria principal para traer un proceso que esté listo para ejecutarse.
Otra razón del sistema operativo	El sistema operativo puede suspender un proceso de fondo o de utilidad o un proceso que se sospecha puede causar un problema.

Motivos de la suspensión de un proceso	
Solicitud interactiva del usuario	Un usuario puede desear suspender la ejecución de un programa con fines de depuración o en relación con el uso de un recurso.
Temporización	Un proceso puede ejecutarse periódicamente (por ejemplo, un proceso de contabilidad o de supervisión del sistema de un sistema) y puede ser suspendido mientras se espera el siguiente intervalo de tiempo.
Solicitud de proceso del padre	Un proceso padre puede querer suspender la ejecución de un descendiente para examinar o modificar el proceso suspendido, o para coordinar la actividad de varios descendientes.

Otro conjunto de razones se refiere a las acciones de un usuario interactivo. Por ejemplo, si un usuario sospecha de un error en el programa, puede depurar el programa suspendiendo su ejecución, examinando y modificando el programa o los datos, y reanudando la ejecución. O puede haber un proceso en segundo plano que esté recopilando estadísticas de rastreo o contabilidad, que el usuario puede querer activar y desactivar.

Las consideraciones de tiempo también pueden llevar a una decisión de intercambio. Por ejemplo, si un proceso debe ser activado periódicamente, pero está inactivo la mayor parte del tiempo, entonces debe ser intercambiado entre usos. Un programa que monitoriza la utilización o la actividad del usuario es un ejemplo.

Finalmente, un proceso padre puede querer suspender un proceso descendiente. Por ejemplo, el proceso A puede generar el proceso B para realizar una lectura de un archivo. Posteriormente, el proceso B encuentra un error en el procedimiento de lectura del archivo e informa de ello al proceso A. El proceso A suspende el proceso B para investigar la causa.

En todos estos casos, la activación de un proceso suspendido es solicitada por el agente que inicialmente solicitó la suspensión.

Estructuras de control del sistema operativo

El sistema operativo controla los eventos dentro del sistema informático. Programa y envía procesos para su ejecución por el procesador, asigna recursos a los procesos y responde a las peticiones de servicios básicos de los procesos de usuario. Fundamentalmente, podemos pensar en el sistema operativo como la entidad que gestiona el uso de los recursos del sistema por parte de los procesos. Este concepto se ilustra en la figura 10.

En un entorno de multiprogramación, hay una serie de procesos ($P_1, \dots, P_n$) que se han creado y existen en la memoria virtual. Cada proceso, durante el curso de su ejecución, necesita acceder a ciertos recursos del sistema, incluyendo el procesador, los dispositivos de E/S y la memoria principal. En la figura, el proceso P_1 se está ejecutando; al menos una parte del proceso está en la memoria principal, y tiene el control de dos dispositivos de E/S. El proceso P_2 también está en la memoria principal, pero está bloqueado esperando un dispositivo de E/S asignado a P_1 . El proceso P_n ha sido intercambiado y, por tanto, está suspendido.

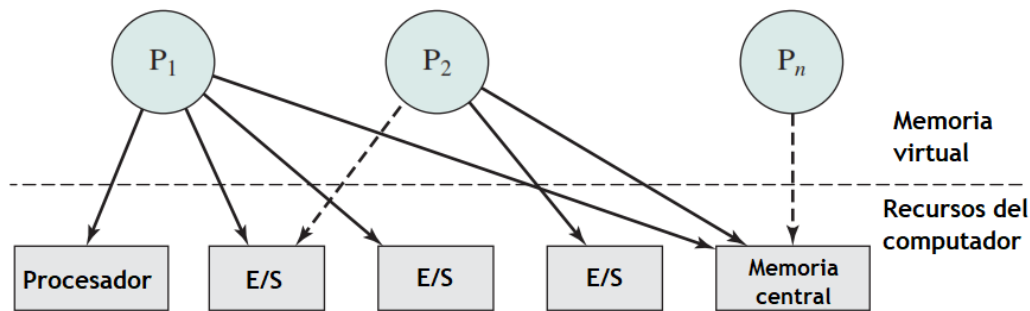


Figura 10

Exploraremos los detalles de la gestión de estos recursos por parte del SO en nombre de los procesos en capítulos posteriores. Aquí nos preocupa una cuestión más fundamental: Puesto que el sistema operativo debe controlar todo lo que ocurre dentro de la computadora, ¿qué información necesita el sistema operativo para controlar los procesos y gestionar los recursos para ellos?

Si el sistema operativo debe gestionar procesos y recursos, debe tener información sobre el estado actual de cada proceso y recurso. El enfoque universal para proporcionar esta información es sencillo: El sistema operativo construye y mantiene tablas de información sobre cada entidad que está gestionando. Una idea general del alcance de este esfuerzo se indica en la figura 11, que muestra cuatro tipos diferentes de tablas mantenidas por el sistema operativo: memoria, E/S, archivo y proceso. Aunque los detalles diferirán de un sistema operativo a otro, fundamentalmente, todos los sistemas operativos mantienen información en estas cuatro categorías.

Las tablas de memoria se utilizan para llevar la cuenta de la memoria principal (real) y secundaria (virtual). Una parte de la memoria principal está reservada para su uso por el sistema operativo; el resto está disponible para su uso por los procesos. Los procesos se mantienen en la memoria secundaria utilizando algún tipo de memoria virtual o un simple mecanismo de intercambio. Las tablas de memoria deben incluir la siguiente información:

- La asignación de memoria principal a los procesos
- La asignación de memoria secundaria a los procesos
- Cualquier atributo de protección de bloques de memoria principal o virtual, como por ejemplo qué procesos pueden acceder a determinadas regiones de memoria compartida
- Cualquier información necesaria para gestionar la memoria virtual

El sistema operativo debe utilizar las tablas de E/S para gestionar los dispositivos de E/S y los módulos de E/S del computador. Pero, en un instante determinado, un dispositivo E/S puede estar disponible o asignado a un proceso en particular. Si la operación de E/S se está realizando, el sistema operativo necesita conocer el estado de la operación y la dirección de memoria principal del área usada como fuente o destino de la transferencia de E/S.

El sistema operativo también puede mantener tablas de archivos. Estas tablas proporcionan información sobre la existencia de archivos, su ubicación en la memoria secundaria, su estado actual y otros atributos. Gran parte de esta información, si no toda, puede ser mantenida y utilizada por un sistema de gestión de archivos, en cuyo caso el sistema operativo tiene poco o

ningún conocimiento de los archivos. En otros sistemas operativos, gran parte de los detalles de la gestión de archivos son gestionados por el propio sistema operativo.

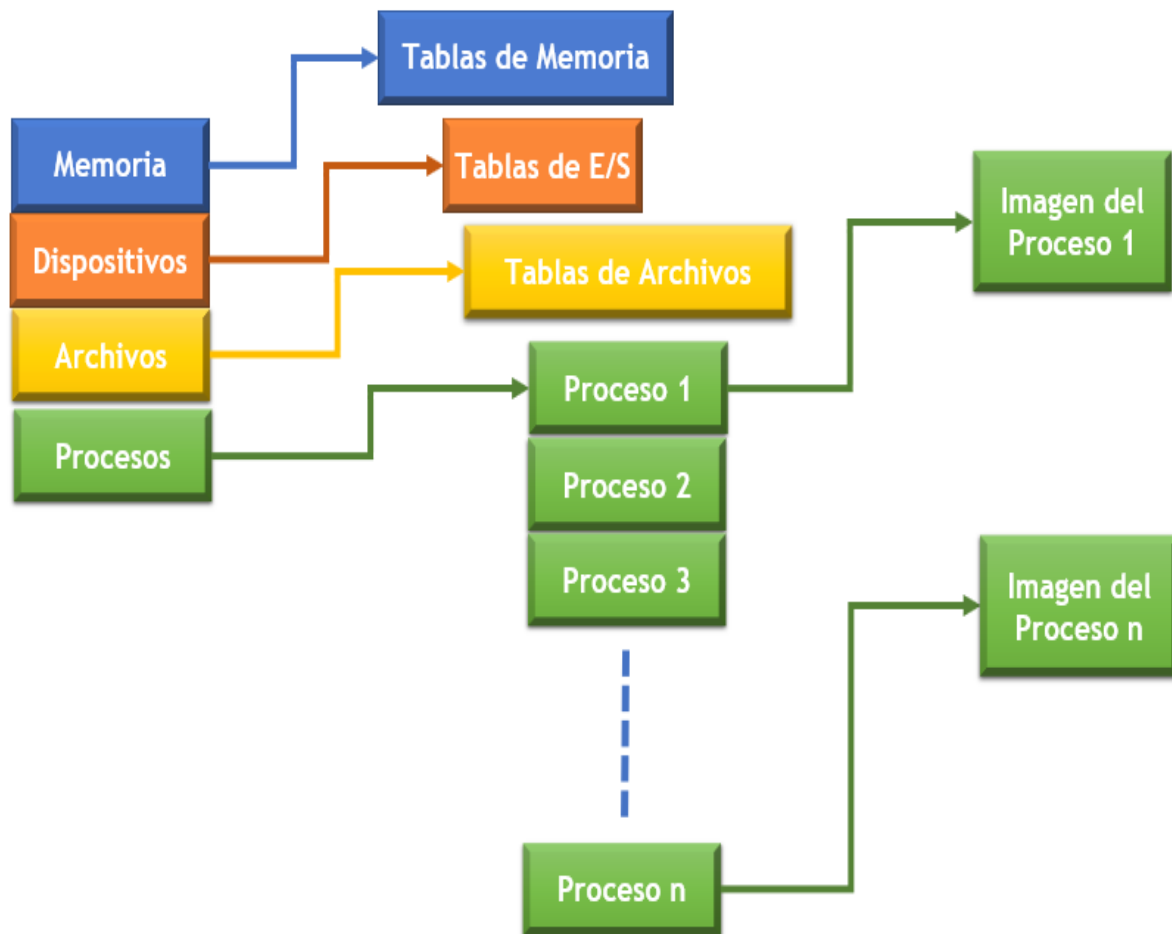


Figura 11

Por último, el sistema operativo debe mantener tablas de procesos para gestionar los procesos. El resto de esta sección se dedica a examinar las tablas de procesos necesarias. Antes de continuar con esto, debemos ver dos puntos adicionales. Primero, aunque la figura 11 muestra cuatro conjuntos distintos de tablas, debe quedar claro que estas tablas deben estar enlazadas o con referencias cruzadas de alguna manera. La memoria, la E/S y los archivos se gestionan en nombre de los procesos, por lo que debe haber alguna referencia a estos recursos, directa o indirectamente, en las tablas de procesos. Los archivos a los que se hace referencia en las tablas de archivos son accesibles a través de un dispositivo de E/S y, en algunas ocasiones, estarán en la memoria principal o virtual. Las propias tablas deben ser accesibles por el sistema operativo y, por tanto, están sujetas a la gestión de la memoria.

En segundo lugar, ¿cómo sabe el sistema operativo crear las tablas en primer lugar? Claramente, el sistema operativo debe tener algún conocimiento del entorno básico, como cuánta memoria principal existe, cuáles son los dispositivos de E/S y cuáles son sus identificadores, etc. Este es un problema de configuración. Es decir, cuando el sistema operativo se inicializa, debe tener acceso a algunos datos de configuración que definen el entorno básico, y estos datos deben ser creados fuera del sistema operativo, con ayuda humana o por algún software de autoconfiguración.

Estructuras de control de proceso. Bloque de control de proceso.

Consideremos lo que el sistema operativo debe saber para gestionar y controlar un proceso. En primer lugar, debe saber dónde se encuentra el proceso; en segundo lugar, debe conocer los atributos del proceso que son necesarios para su gestión, por ejemplo, la identificación y el estado del proceso.

Localización de los procesos

Antes de abordar las cuestiones de dónde se encuentra un proceso o cuáles son sus atributos, tenemos que abordar una cuestión aún más fundamental: ¿Cuál es la manifestación física de un proceso? Como mínimo, un proceso debe incluir un programa o conjunto de programas que se ejecutan. Asociado a estos programas hay un conjunto de ubicaciones de datos para las variables locales y globales y cualquier constante definida. Por lo tanto, un proceso consistirá en al menos suficiente memoria para mantener los programas y los datos de ese proceso. Además, la ejecución de un programa suele implicar una pila que se utiliza para llevar la cuenta de las llamadas a procedimientos y el paso de parámetros entre procedimientos. Finalmente, cada proceso tiene asociado un número de atributos que son utilizados por el sistema operativo para el control del proceso. Normalmente, la colección de atributos se denomina *bloque de control de procesos*. Otros nombres habituales usados para esta estructura de datos son bloque de control de tarea, descriptor del proceso, y descriptor de tarea. Podemos referirnos a esta colección de programa, datos, pila y atributos como la *imagen del proceso* (Tabla *Elementos típicos de la imagen de un proceso*).

Elementos típicos de la imagen de un proceso	
Datos del usuario	La parte modificable del espacio de usuario. Puede incluir datos de programa, área de pila de usuario, y programas que puedan ser modificados.
Programa de usuario	El programa por ejecutar.
Pila de sistema	Cada proceso tiene una o más pilas de sistema (LIFO) asociadas a él. Una pila se utiliza para almacenar los parámetros y las direcciones de retorno de los procedimientos y llamadas a sistema.
Bloque de control de proceso	Datos necesarios para que el sistema operativo pueda controlar los procesos (Tabla <i>Elementos típicos de un bloque de control del proceso (BCP)</i>).

La ubicación de una imagen de proceso dependerá del esquema de gestión de memoria que se utilice. En el caso más sencillo, la imagen del proceso se mantiene como un bloque de memoria contiguo o continuo. Este bloque se mantiene en la memoria secundaria, normalmente en el disco. Para que el sistema operativo pueda gestionar el proceso, al menos una pequeña parte de su imagen debe mantenerse en la memoria principal. Para ejecutar el proceso, toda la imagen del proceso debe cargarse en la memoria principal o al menos en la memoria virtual. Por lo tanto, el sistema operativo necesita conocer la ubicación de cada proceso en el disco y, para cada uno de esos procesos que está en la memoria principal, la ubicación de ese proceso en la

memoria principal. Una variación ligeramente más compleja de este esquema la presentan aquellos sistemas operativos que, cuando un proceso es intercambiado, parte de la imagen del proceso puede permanecer en la memoria principal. Por lo tanto, el sistema operativo debe llevar la cuenta de qué partes de la imagen de cada proceso están todavía en la memoria principal.

Los sistemas operativos modernos suponen un hardware de paginación que permite una memoria física no contigua para soportar procesos parcialmente residentes. En cualquier momento, una porción de la imagen de un proceso puede estar en la memoria principal, y el resto en la memoria secundaria. Esta breve discusión pasa por alto algunos detalles. En particular, en un sistema que utiliza memoria virtual, toda la imagen de un proceso activo está siempre en la memoria secundaria. Cuando una parte de la imagen se carga en la memoria principal, se copia en lugar de moverse. Así, la memoria secundaria conserva una copia de todos los segmentos y/o páginas. Sin embargo, si la porción de la imagen en la memoria principal se modifica, la copia secundaria estará desactualizada hasta que la porción de la memoria principal se vuelva a copiar en el disco. Por lo tanto, las tablas de procesos mantenidas por el sistema operativo deben mostrar la ubicación de cada página de cada imagen de proceso.

La figura 11 muestra la estructura de la información de localización de la siguiente manera. Hay una tabla de procesos primaria con una entrada para cada proceso. Cada entrada contiene, al menos, un puntero a una imagen de proceso. Si la imagen del proceso contiene múltiples bloques, esta información está contenida directamente en la tabla de procesos primaria o está disponible por referencia cruzada a las entradas en las tablas de memoria. Por supuesto, esta representación es genérica; un sistema operativo concreto tendrá su propia forma de organizar la información de localización.

Atributos de proceso

Un sofisticado sistema de multiprogramación requiere una gran cantidad de información sobre cada proceso. Como se ha explicado, se puede considerar que esta información reside en un bloque de control de procesos. Diferentes sistemas organizarán esta información de diferentes maneras, y varios ejemplos de esto aparecen al final de este capítulo y el siguiente. Por ahora, simplemente exploremos el tipo de información que podría ser de utilidad para un sistema operativo sin considerar en detalle cómo se organiza esa información.

La tabla *Elementos típicos de un bloque de control del proceso (BCP)* enumera las categorías típicas de información requeridas por el sistema operativo para cada proceso. Es posible que se sorprenda de la cantidad de información que se necesita. A medida que usted adquiere una mayor apreciación de las responsabilidades del sistema operativo, esta lista debería parecer más razonable. Podemos agrupar la información del bloque de control del proceso en tres categorías generales:

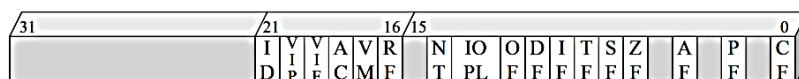
- Identificación del proceso
- Información del estado del procesador
- Información de control del proceso

Con respecto a la identificación de procesos, en prácticamente todos los sistemas operativos, a cada proceso se le asigna un identificador numérico único, que puede ser simplemente un

índice en la tabla de procesos primarios (Figura 11); de lo contrario, debe haber un mapeo que permita al sistema operativo localizar las tablas apropiadas basándose en el identificador del proceso. Este identificador es útil de varias maneras. Muchas de las otras tablas controladas por el sistema operativo pueden usar identificadores de procesos para hacer referencias cruzadas a las tablas de procesos. Por ejemplo, las tablas de memoria pueden estar organizadas para proporcionar un mapa de la memoria principal con una indicación de qué proceso está asignado a cada región. Referencias similares aparecerán en las tablas de E/S y de archivos. Cuando los procesos se comunican entre sí, el identificador del proceso informa al sistema operativo del destino de una comunicación concreta. Cuando los procesos pueden crear otros procesos, los identificadores indican el padre y los descendientes de cada proceso. Además de estos identificadores de proceso, a un proceso se le puede asignar un identificador de usuario que indica el usuario responsable del trabajo.

La información del estado del procesador consiste en el contenido de los registros del procesador. Mientras un proceso está en marcha, por supuesto, la información está en los registros. Cuando un proceso se interrumpe, toda esta información de los registros debe ser guardada para que pueda ser restaurada cuando el proceso reanude su ejecución. La naturaleza y el número de registros implicados dependen del diseño del procesador. Normalmente, el conjunto de registros incluirá registros visibles para el usuario, registros de control y de estado, y punteros de pila.

Cabe destacar que todos los diseños de procesadores incluyen un registro o conjunto de registros, a menudo conocido como palabra de estado del programa (PSW), que contiene información de estado. La PSW suele contener códigos de condición además de otra información de estado. Un buen ejemplo de una palabra de estado del procesador es el de los procesadores Intel x86, conocido como el registro EFLAGS, mostrado en la figura 12. Esta estructura es utilizada por cualquier sistema operativo (incluyendo UNIX y Windows) que se ejecute en un procesador x86.



- | | |
|---|--|
| ID = (Indicador de identificación) | DF = (Indicador de dirección) |
| VIP = (Interrupción virtual pendiente) | IF = (Indicador de interrupciones habilitadas) |
| VIF = (Indicador de interrupción virtual) | TF = (Indicador de <i>trap</i>) |
| AC = (Comprobación de alineamiento) | SF = (Indicador de signo) |
| VM = (Modo virtual 8086) | ZF = (Indicador de cero) |
| RF = (Indicación de reanudación) | AF = (Indicador auxiliar de acarreo) |
| NT = (Indicador de tarea anidada) | PF = (Indicador de paridad) |
| IOPL = (Nivel de privilegio de E/S) | CF = (Indicador de acarreo) |
| OF = (Indicador de desbordamiento) | |

Figura 12: Registro EFLAGS de un Pentium II.

La tercera categoría principal de información en el bloque de control de procesos puede llamarse, a falta de un nombre mejor, información de control de procesos. Es la información adicional que necesita el sistema operativo para controlar y coordinar los distintos procesos activos. La última parte de la tabla *Elementos típicos de un bloque de control del proceso (BCP)* indica el alcance de esta información. A medida que examinemos los detalles de la funcionalidad del sistema operativo en los siguientes módulos, la necesidad de los distintos elementos de esta lista debería quedar clara.

Elementos típicos de un bloque de control del proceso (BCP)	
Identificación del proceso	
Identificadores	Identificadores numéricos que se pueden guardar dentro del bloque de control del proceso: ➤ Identificadores del proceso ➤ Identificador del proceso que creó a este proceso (proceso padre) ➤ Identificador del usuario
Información de estado del procesador	
Registros visibles por el usuario	Un registro visible por el usuario es aquel al que se puede hacer referencia por medio del lenguaje máquina que ejecuta el procesador cuando está en modo usuario. Normalmente, existen de 8 a 32 de estos registros, aunque determinadas implementaciones RISC tienen más de 100.
Registros de estado control	Hay una gran variedad de registros del procesador que se utilizan para el control de operaciones. Estos incluyen: ➤ Contador de programa: contiene la dirección de la siguiente instrucción a ejecutar. ➤ Códigos de condición: resultan de la operación lógica o aritmética más reciente (por ejemplo, signo, cero, acarreo, igual, desbordamiento). ➤ Información de estado: incluyen los flags de interrupciones habilitadas/deshabilitadas, modo ejecución.
Puntero de pila	Cada proceso tiene una o más pilas de sistema (LIFO) asociadas a él. Una pila se utiliza para almacenar los parámetros y las direcciones de retorno de los procedimientos y llamadas a sistema. Un puntero de pila apunta a la parte más alta de la pila.
Información de control de proceso	
Información de estado y de planificación	Esta es la información que el sistema operativo necesita para analizar las funciones de planificación. Elementos típicos de esta información son: ➤ Estado del proceso: indica si está listo o no el proceso para ser planificado para su ejecución (por ejemplo, Ejecutando, Listo, Esperando, Detenido). ➤ Prioridad: uno o más campos que se pueden utilizar para escribir la prioridad de planificación del proceso. En algunos sistemas, se necesitan múltiples valores (por ejemplo, por-defecto, actual, mayor-disponible). ➤ Información relativa a planificación: esto dependerá del algoritmo de planificación utilizado. Por ejemplo, la cantidad de tiempo que el proceso estaba esperando y la cantidad de tiempo que el proceso ha ejecutado la última vez que estuvo corriendo. ➤ Evento: identificar el evento al cual el proceso está esperando para continuar su ejecución.
Estructuración de datos	Un proceso puede estar enlazado con otro proceso en una cola, anillo o con cualquier otra estructura. Por ejemplo, todos los procesos en estado de

Elementos típicos de un bloque de control del proceso (BCP)	
	espera por un nivel de prioridad en particular pueden estar enlazados en una cola. Un proceso puede mostrar una relación padre-hijo (creador-creado) con otro proceso. El bloque de control del proceso puede contener punteros a otros procesos para dar soporte a estas estructuras.
Comunicación entre procesos	Se pueden asociar diferentes flags, señales, y mensajes relativos a la comunicación entre dos procesos independientes. Alguna o toda esta información se puede mantener en el bloque de control de proceso (BCP).
Privilegios de proceso	Los procesos adquieren privilegios de acuerdo con la memoria a la que van a acceder y los tipos de instrucciones que se pueden ejecutar. Adicionalmente, los privilegios se pueden utilizar para usar utilidades de sistema o servicios.
Gestión de memoria	Esta sección puede incluir punteros a tablas de segmentos y/o páginas que describen la memoria virtual asignada a este proceso.
Propiedad y utilización de los recursos	Se deben indicar los recursos controlados por el proceso, por ejemplo, archivos abiertos. También se puede incluir un histórico de utilización del procesador o de otros recursos; esta información puede ser necesaria para el planificador.

La figura 13 sugiere la estructura de las imágenes de proceso en la memoria virtual. Cada imagen de proceso consiste en un bloque de control de proceso, una pila de usuario, el espacio de direcciones privado del proceso y cualquier otro espacio de direcciones que el proceso comparta con otros procesos. En la figura, cada imagen de proceso aparece como un rango contiguo de direcciones. En una implementación real, este puede no ser el caso; dependerá del esquema de gestión de memoria y de la forma en que las estructuras de control sean organizadas por el sistema operativo.

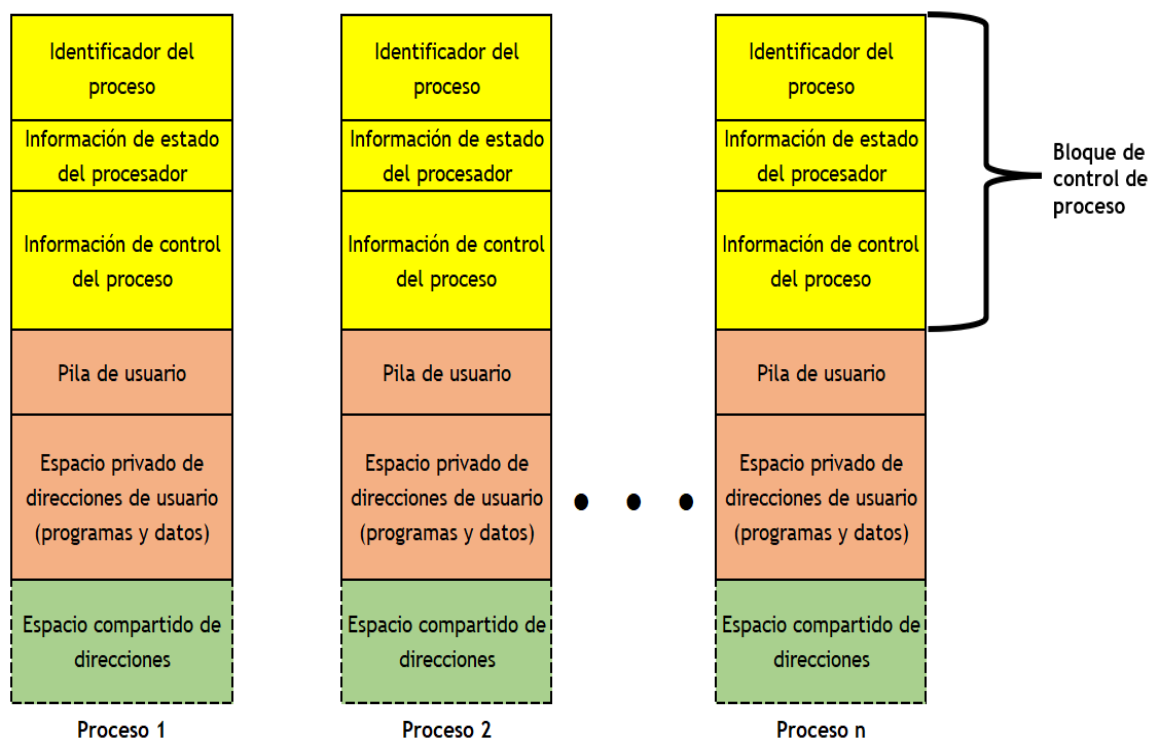


Figura 13: Procesos de usuario en memoria virtual.

Como se indica en la tabla *Elementos típicos de un bloque de control del proceso (BCP)*, el bloque de control del proceso puede contener información de estructuración, incluyendo punteros que permiten la vinculación de los bloques de control del proceso. Así, las colas que se describieron en la sección anterior podrían implementarse como listas enlazadas de bloques de control de procesos. Por ejemplo, la estructura de colas de la figura 8(a) podrían implementarse como se sugiere en la figura 14.

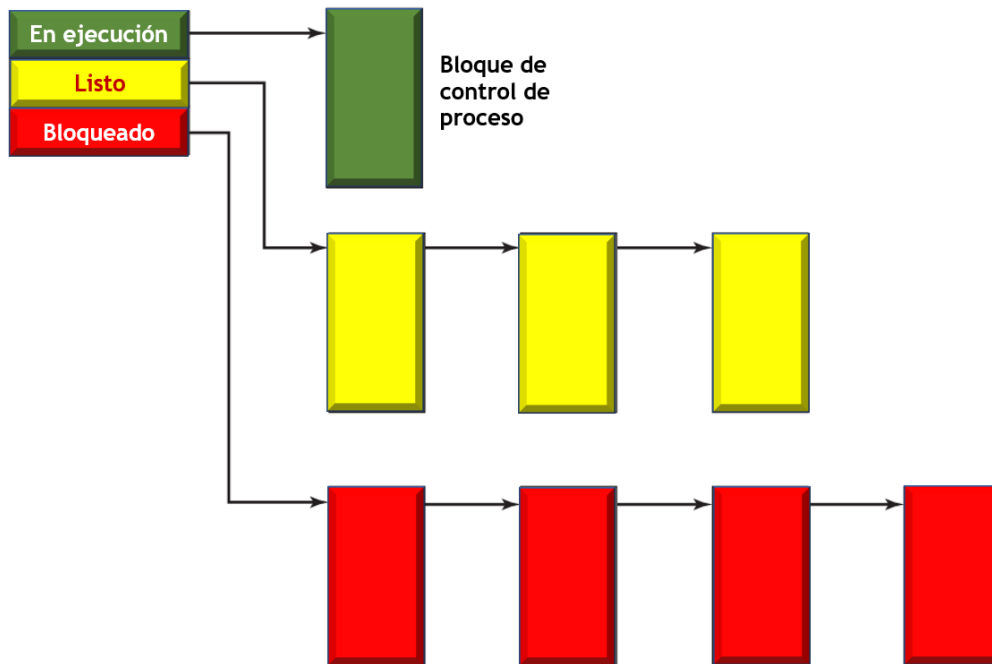


Figura 14: Estructuras de listas de procesos.

El papel del bloque de control de procesos

El bloque de control de procesos es la estructura de datos más importante de un sistema operativo. Cada bloque de control de procesos contiene toda la información sobre un proceso que necesita el sistema operativo. Los bloques son leídos y/o modificados por prácticamente todos los módulos del sistema operativo, incluidos los que se ocupan de la programación, la asignación de recursos, el procesamiento de interrupciones y la supervisión y el análisis del rendimiento. Se puede decir que el conjunto de bloques de control de procesos define el estado del sistema operativo.

Esto plantea una importante cuestión de diseño. Varias rutinas dentro del sistema operativo necesitarán acceder a la información de los bloques de control de procesos. La provisión de acceso directo a estas tablas no es difícil. Cada proceso está equipado con un identificador único, y esto puede ser utilizado como un índice en una tabla de punteros a los bloques de control de procesos. La dificultad no es el acceso, sino la protección. Se presentan dos problemas:

- Un fallo en una sola rutina, como un manejador de interrupciones, podría dañar los bloques de control de procesos, lo que podría destruir la capacidad del sistema para gestionar los procesos afectados.
- Un cambio de diseño en la estructura o la semántica del bloque de control de procesos podría afectar a varios módulos del sistema operativo. Estos problemas pueden

solucionarse requiriendo que todas las rutinas del sistema operativo pasen por una rutina manejadora, cuyo único trabajo es proteger los bloques de control de procesos, y que es el único árbitro para leer y escribir estos bloques. La compensación en el uso de tal rutina implica cuestiones de rendimiento y el grado en que se puede confiar en que el resto del software del sistema sea correcto.

En definitiva, lo importante del bloque de control de procesos es que contiene información suficiente para que sea posible interrumpir un proceso en ejecución y reanudar posteriormente la ejecución como si no se hubiera producido la interrupción. El bloque de control de procesos es la herramienta clave que permite al sistema operativo soportar múltiples procesos y proporcionar multiprocesamiento. Cuando se interrumpe un proceso, los valores actuales del contador de programa y los registros del procesador (datos de contexto) se guardan en los campos apropiados del bloque de control de procesos correspondiente, y el estado del proceso se cambia a algún otro valor, como bloqueado o listo (descrito posteriormente). El sistema operativo está ahora libre para poner algún otro proceso en el estado de ejecución. El contador de programa y los datos de contexto para este proceso se cargan en los registros del procesador y este proceso comienza a ejecutarse. Por lo tanto, podemos decir que un proceso consiste en el código del programa y los datos asociados más un bloque de control del proceso. En un computador con un solo procesador, en un momento dado, se está ejecutando como máximo un proceso y ese proceso está en estado de ejecución.

Acciones con procesos

Con respecto a los procesos es posible realizar las siguientes acciones:

- Crearlo.
- Destruirlo.
- Modificarle la prioridad.
- Bloquear su ejecución.
- Desbloquear su ejecución.
- Suspenderlo.
- Activarlo.

Tipos de procesos

Se puede clasificar a los procesos en las siguientes categorías, no todas ellas presentes en todos los sistemas operativos:

- Intercambiables. Son los que, durante su ejecución, el sistema operativo puede en un swapping llevar a disco.
- Residentes. Procesos que siempre durante su ejecución deben residir totalmente en memoria central.
- Reutilizables. Son aquellos que deben ser cargados en memoria siempre que son ejecutados, pudiéndoselos usar todas las veces que se quiera como es el caso de los programas de usuario.

- Reentrantes. Son aquellos que sólo necesitan tener una copia del código en memoria cuando más de un usuario los están usando y, por ende, para cada uno de esos usuarios se crea un proceso.
- Padre. Es aquel proceso que crea a otro proceso.
- Hijo. Es el proceso que es creado por otro proceso.
- Privilegiado. Proceso que es ejecutado en modo kernel.
- No privilegiado. Proceso que se ejecuta en modo usuario.
- De usuario. Proceso creado por el usuario que se ejecuta en modo usuario.
- Del sistema. Son los creados por el sistema operativo.
- Apropiativos. Son los que una vez que tienen los recursos que necesitan, sólo los dejan voluntariamente.
- No apropiativos. Son procesos que les permiten a otros procesos sacarles los recursos que tienen asignados.

Control de procesos

Modos de ejecución

Antes de continuar con la forma en que el sistema operativo gestiona los procesos, necesitamos distinguir entre el modo de ejecución del procesador normalmente asociado con el sistema operativo y el normalmente asociado con los programas de usuario. La mayoría de los procesadores soportan al menos dos modos de ejecución. Algunas instrucciones sólo pueden ejecutarse en el modo más privilegiado. Estas incluyen la lectura o alteración de un registro de control, como la palabra de estado del programa; instrucciones primitivas de E/S; e instrucciones relacionadas con la gestión de la memoria. Además, sólo se puede acceder a ciertas regiones de la memoria en el modo con más privilegios.

El modo menos privilegiado se denomina a menudo *modo de usuario* porque los programas de usuario suelen ejecutarse en este modo. El modo más privilegiado se denomina *modo sistema*, *modo control* o *modo kernel*. Este último término se refiere al núcleo del sistema operativo, que es la parte del sistema operativo que abarca las funciones importantes del sistema. La razón para utilizar dos modos debería estar clara. Es necesario proteger el sistema operativo y las tablas clave del sistema operativo, como los bloques de control de procesos, de la interferencia de los programas de usuario. En el modo kernel, el software tiene el control completo del procesador y de todas sus instrucciones, registros y memoria. Este nivel de control no es necesario y, por seguridad, no es deseable para los programas de usuario.

Surgen dos preguntas: ¿Cómo sabe el procesador en qué modo debe ejecutarse y cómo se cambia el modo? Respecto a la primera pregunta, normalmente hay un bit en la palabra de estado del programa (PSW) que indica el modo de ejecución. Este bit se cambia en respuesta a ciertos eventos. Normalmente, cuando un usuario hace una llamada a un servicio del sistema operativo o cuando una interrupción desencadena la ejecución de una rutina del sistema operativo, el modo se establece en el modo del núcleo y, al volver del servicio al proceso de

usuario, el modo se establece en el modo de usuario. Como ejemplo, consideremos al procesador Intel Itanium, que implementa la arquitectura IA-64 de 64 bits. El procesador tiene un registro de estado del procesador (PSR: Processor Status Register) que incluye un campo llamado nivel de privilegio actual (CPL: Current Privilege Level) de 2 bits. El nivel 0 es el más privilegiado, mientras que el nivel 3 es el menos privilegiado. La mayoría de los sistemas operativos, como Linux, utilizan el nivel 0 para el núcleo y otro nivel para el modo de usuario. Cuando se produce una interrupción, el procesador borra la mayoría de los bits del PSR, incluyendo el campo CPL. Esto automáticamente pone el CPL en el nivel 0. Al final de la rutina de manejo de interrupciones, la última instrucción que se ejecuta es la de retorno de la interrupción (IRT: Interrupt Return). Esta instrucción hace que el procesador restaure el PSR del programa interrumpido, lo que restaura el nivel de privilegio de ese programa. Una secuencia similar ocurre cuando una aplicación realiza una llamada al sistema. Para el Itanium, una aplicación realiza una llamada al sistema colocando el identificador de la llamada al sistema y los argumentos de la llamada al sistema en un área predefinida y luego ejecutando una instrucción especial que tiene el efecto de interrumpir la ejecución en el nivel de usuario y transferir el control al núcleo.

Creación de procesos

Antes discutimos los eventos que llevan a la creación de un nuevo proceso. Habiendo discutido las estructuras de datos asociadas con un proceso, ahora podemos describir brevemente los pasos involucrados en la creación del proceso. Una vez que el sistema operativo decide, por la razón que sea, crear un nuevo proceso, puede proceder como sigue:

- **Asignar un identificador de proceso único al nuevo proceso.** En este momento, se añade una nueva entrada a la tabla de procesos primaria, que contiene una entrada por proceso.
- **Asignar espacio para el proceso.** Esto incluye todos los elementos de la imagen del proceso. Así, el sistema operativo debe saber cuánto espacio se necesita para el espacio de direcciones privado del usuario (programas y datos) y la pila del usuario. Estos valores pueden ser asignados por defecto en función del tipo de proceso, o pueden ser fijados en función de la petición del usuario en el momento de la creación del trabajo. Si un proceso es generado por otro proceso, el proceso padre puede pasar los valores necesarios al sistema operativo como parte de la solicitud de creación del proceso. Si algún espacio de direcciones existente va a ser compartido por este nuevo proceso, deben establecerse los enlaces apropiados. Por último, se debe asignar espacio para un bloque de control del proceso.
- **Inicializar el bloque de control del proceso.** La parte de identificación del proceso contiene el ID de este proceso más otros ID's apropiados, como el del proceso padre. La porción de información del estado del procesador se inicializará normalmente con la mayoría de las entradas a cero, excepto el contador de programa (establecido en el punto de entrada del programa) y los punteros de la pila del sistema (establecidos para definir los límites de la pila del proceso). La parte de información de control del proceso se inicializa basándose en los valores estándar por defecto más los atributos que se han solicitado para este proceso. Por ejemplo, el estado del proceso se inicializa típicamente

a Listo o Listo/Suspendido. La prioridad puede establecerse por defecto en la prioridad más baja, a menos que se solicite explícitamente una prioridad más alta. Inicialmente, el proceso puede no poseer recursos (dispositivos de E/S, archivos) a menos que haya una petición explícita de estos o a menos que sean heredados del padre.

- **Establecer los enlaces apropiados.** Por ejemplo, si el sistema operativo mantiene cada cola de programación como una lista enlazada, entonces el nuevo proceso debe ser puesto en la lista de Listos o de Listos/Suspendidos.
- **Crear o ampliar otras estructuras de datos.** Por ejemplo, el sistema operativo puede mantener un archivo de contabilidad en cada proceso que se utilizará posteriormente para fines de facturación y/o evaluación del rendimiento del sistema.

Cambio de proceso

A primera vista, la función del cambio de proceso parece ser sencilla. En algún momento, un proceso en ejecución se interrumpe, y el sistema operativo asigna otro proceso al estado de Ejecución y le entrega el control a ese proceso. Sin embargo, se plantean varias cuestiones de diseño. En primer lugar, ¿qué eventos desencadenan un cambio de proceso? Otra cuestión es que debemos reconocer la distinción entre el cambio de modo y el cambio de proceso. Por último, ¿qué debe hacer el sistema operativo con las diversas estructuras de datos bajo su control para lograr un cambio de proceso?

¿Cuándo cambiar de proceso?

Un cambio de proceso puede ocurrir en cualquier momento que el sistema operativo haya obtenido el control del proceso que se está ejecutando actualmente. La tabla *Mecanismos para la interrupción de la ejecución de un proceso* sugiere los posibles eventos que pueden dar el control al sistema operativo.

Mecanismos para la interrupción de la ejecución de un proceso		
Mecanismo	Causa	Uso
Interrupción	Externa a la ejecución del proceso actualmente en ejecución.	Reacción ante un evento externo asíncrono.
Trap	Asociada a la ejecución de la instrucción actual.	Manejo de una condición de error o de excepción.
Llamada al sistema	Solicitud explícita.	Llamada a una función del sistema operativo.

Primero, consideremos las interrupciones del sistema. En realidad, podemos distinguir, como hacen muchos sistemas, dos tipos de interrupciones del sistema, una de las cuales se denomina simplemente interrupción y la otra trampa. La primera se debe a algún tipo de evento externo e independiente del proceso que se está ejecutando, como la finalización de una operación de E/S. La segunda se relaciona con una condición de error o excepción generada dentro del proceso que se está ejecutando, como un intento de acceso ilegal a un archivo. Con una interrupción ordinaria, el control se transfiere primero a un manejador de interrupciones, que

hace algunas tareas básicas y luego se bifurca a una rutina del sistema operativo que se ocupa del tipo particular de interrupción que ha ocurrido. Algunos ejemplos son los siguientes:

- **Interrupción del reloj:** El sistema operativo determina si el proceso que se está ejecutando en ese momento lo ha hecho durante la unidad de tiempo máxima permitida, denominada porción de tiempo. Es decir, un intervalo de tiempo es la cantidad máxima de tiempo que un proceso puede ejecutar antes de ser interrumpido. Si es así, este proceso debe pasar a un estado Listo y se debe enviar otro proceso.
- **Interrupción de E/S:** El sistema operativo determina qué acción de E/S ha ocurrido. Si la acción de E/S constituye un evento por el cual uno o más procesos están esperando, entonces el sistema operativo mueve todos los procesos bloqueados correspondientes al estado Listo (y los procesos Bloqueados/Suspendidos al estado Listo/Suspendido). El sistema operativo debe entonces decidir si reanudar la ejecución del proceso que se encuentra en el estado de Ejecución o si prefiere sustituirlo por un proceso Listo de mayor prioridad.
- **Fallo de memoria:** El procesador encuentra una referencia de dirección de memoria virtual para una palabra que no está en la memoria principal. El sistema operativo debe traer el bloque (página o segmento) de memoria que contiene la referencia de la memoria secundaria a la memoria principal. Después de que se emita la solicitud de E/S para traer el bloque de memoria, el proceso con el fallo de memoria se coloca en un estado bloqueado; el sistema operativo entonces realiza un cambio de proceso para reanudar la ejecución de otro proceso. Después de que el bloque deseado es traído a la memoria, ese proceso es colocado en el estado Listo.

Con una trampa, el sistema operativo determina si la condición de error o excepción es fatal. Si es así, entonces el proceso que se está ejecutando actualmente se mueve al estado Terminado y se produce un cambio de proceso. Si no, la acción del sistema operativo dependerá de la naturaleza del error y del diseño del sistema operativo. Puede intentar algún procedimiento de recuperación o simplemente notificar al usuario. Puede hacer un cambio de proceso o reanudar el proceso que se está ejecutando actualmente.

Finalmente, el sistema operativo puede ser activado por una llamada al supervisor desde el programa que se está ejecutando. Por ejemplo, un proceso de usuario se está ejecutando, y se ejecuta una instrucción que solicita una operación de E/S, como la apertura de un archivo. Esta llamada resulta en una transferencia a una rutina que es parte del código del sistema operativo. El uso de una llamada al sistema puede colocar al proceso de usuario en el estado de Bloqueo.

Cambio de modo

En el módulo 1, vimos la inclusión de una etapa de interrupción como parte del ciclo de instrucción. Recordemos que, en la etapa de interrupción, el procesador comprueba si hay alguna interrupción pendiente, indicada por la presencia de una señal de interrupción. Si no hay interrupciones pendientes, el procesador pasa a la etapa de obtención y obtiene la siguiente instrucción del programa actual en el proceso actual. Si hay una interrupción pendiente, el procesador hace lo siguiente:

- Pone el contador de programa en la dirección de inicio de un programa controlador de interrupciones.
- Pasa del modo usuario al modo kernel para que el código de procesamiento de la interrupción pueda incluir instrucciones privilegiadas.

El procesador procede ahora a la etapa de obtención y obtiene la primera instrucción del programa gestor de la interrupción, que dará servicio a la interrupción. En este punto, normalmente, el contexto del proceso que ha sido interrumpido se guarda en el bloque de control del proceso del programa interrumpido.

Una pregunta que se le puede ocurrir ahora es, ¿qué constituye el contexto que se guarda? La respuesta es que debe incluir cualquier información que pueda ser alterada por la ejecución del manejador de la interrupción y que será necesaria para reanudar el programa que fue interrumpido. Por lo tanto, la parte del bloque de control del proceso que fue referida como información del estado del procesador debe ser guardada. Esto incluye el contador de programa, otros registros del procesador y la información de la pila.

¿Hay que hacer algo más? Eso depende de lo que ocurra a continuación. El manejador de la interrupción es típicamente un programa corto que realiza algunas tareas básicas relacionadas con una interrupción. Por ejemplo, restablece la bandera o el indicador que señala la presencia de una interrupción. Puede enviar un acuse de recibo a la entidad que emitió la interrupción, como un módulo de E/S. Y puede hacer algunas tareas básicas relacionadas con los efectos del evento que causó la interrupción. Por ejemplo, si la interrupción está relacionada con un evento de E/S, el manejador de la interrupción comprobará si hay una condición de error. Si ha ocurrido un error, el manejador de la interrupción puede enviar una señal al proceso que originalmente solicitó la operación de E/S. Si la interrupción es por el reloj, entonces el manejador entregará el control al despachador, que querrá pasar el control a otro proceso porque la porción de tiempo asignada al proceso que se está ejecutando actualmente ha expirado.

¿Qué pasa con la otra información en el bloque de control del proceso? Si esta interrupción va a ser seguida por un cambio a otro proceso, entonces habrá que hacer algún trabajo. Sin embargo, en la mayoría de los sistemas operativos, la ocurrencia de una interrupción no significa necesariamente un cambio de proceso. Es posible que, después de que el manejador de la interrupción se haya ejecutado, el proceso que se está ejecutando actualmente reanude la ejecución. En ese caso, todo lo que se necesita es guardar la información del estado del procesador cuando se produce la interrupción y restaurar esa información cuando se devuelve el control al programa que se estaba ejecutando. Normalmente, las funciones de guardado y restauración se realizan en el hardware.

Cambio de estado del proceso

Queda claro, pues, que el cambio de modo es un concepto distinto al del cambio de proceso. Sin embargo, se debe tener presente que el término cambio de contexto se encuentra a menudo en muchos textos sobre el sistema operativo y que, aunque la mayor parte de ellos utiliza este término para significar lo que aquí se llama un cambio de proceso, otras fuentes lo utilizan para significar un cambio de modo o incluso un cambio de hilo. Para evitar la ambigüedad, el término no se utiliza aquí. Un cambio de modo puede ocurrir sin cambiar el estado del proceso

que está actualmente en el estado de Ejecución. En ese caso, el guardado del contexto y la subsiguiente restauración implican poca sobrecarga. Sin embargo, si el proceso actualmente en ejecución debe ser movido a otro estado (Listo, Bloqueado, etc.), entonces el sistema operativo debe hacer cambios sustanciales en su entorno. Los pasos involucrados en un cambio de proceso completo son los siguientes:

- Guardar el contexto del procesador, incluyendo el contador de programa y otros registros.
- Actualizar el bloque de control de proceso del proceso que está actualmente en el estado de Ejecución. Esto incluye cambiar el estado del proceso a uno de los otros estados (Listo; Bloqueado; Listo/Suspendido; o Terminado). También deben actualizarse otros campos relevantes, incluyendo la razón para dejar el estado de Ejecución y la información contable.
- Mover el bloque de control de proceso de este proceso a la cola apropiada (Listo; Bloqueado en el Evento i; Listo/Suspendido).
- Seleccionar otro proceso para su ejecución.
- Actualizar el bloque de control del proceso seleccionado. Esto incluye cambiar el estado de este proceso a en Ejecución.
- Actualizar las estructuras de datos de gestión de la memoria. Esto puede ser necesario, dependiendo de cómo se gestiona la traducción de direcciones.
- Restaurar el contexto del procesador al que existía en el momento en que el proceso seleccionado fue cambiado por última vez del estado en Ejecución, cargando los valores anteriores del contador de programa y otros registros.

Así, el cambio de proceso, que implica un cambio de estado, requiere más esfuerzo que un cambio de modo.

Hilo

Seguidamente examinaremos algunos conceptos relacionados con la gestión de procesos, que se encuentran en varios sistemas operativos contemporáneos. Hemos comprobado que el concepto de proceso es más complejo y sutil de lo que se ha presentado hasta ahora y, de hecho, encarna dos conceptos separados y potencialmente independientes: uno relativo a la propiedad de los recursos y otro relativo a la ejecución. Esta distinción ha llevado al desarrollo, en muchos sistemas operativos de estructuras conocidas como hilos (threads).

Hasta ahora se ha presentado el concepto de proceso con dos características:

- Propiedad de los recursos: Un proceso incluye un espacio de direcciones virtual para mantener la imagen del proceso; recordemos que la imagen del proceso es la colección de programa, datos, pila y atributos definidos en el bloque de control del proceso. De vez en cuando, a un proceso se le puede asignar el control o la propiedad de los recursos, como la memoria principal, los canales de E/S, los dispositivos de E/S y los archivos. El sistema operativo realiza una función de protección para evitar interferencias no deseadas entre los procesos con respecto a los recursos.

- Programación/ejecución: La ejecución de un proceso sigue una ruta de ejecución (trazado) a través de uno o más programas. Esta ejecución puede intercalarse con la de otros procesos. Así, un proceso tiene un estado (En ejecución, Listo, etc.) y una prioridad de despacho y es la entidad que es programada y despachada por el sistema operativo.

Estas dos características son independientes y podrían ser tratadas independientemente por el sistema operativo. Esto se hace en varios sistemas operativos, en especial en los desarrollados recientemente. Para distinguir las dos características, la unidad de despacho se suele denominar *hilo* o *proceso ligero*, mientras que la unidad de propiedad de recursos se suele denominar *proceso* o *tarea*. Dicho de otra manera, un hilo es una unidad básica de utilización de la CPU; comprende un ID de hilo, un contador de programa (CP), un conjunto de registros y una pila. Comparte con otros hilos pertenecientes al mismo proceso su sección de código, su sección de datos y otros recursos del sistema operativo, como archivos abiertos y señales. Un proceso tradicional tiene un único hilo de control. Si un proceso tiene varios hilos de control, puede realizar más de una tarea a la vez. La figura 15 ilustra la diferencia entre un proceso tradicional de un solo hilo y un proceso multihilo. Por desgracia, ni siquiera este grado de coherencia se mantiene. En los sistemas operativos mainframe de IBM, los conceptos de espacio de direcciones y tarea, respectivamente, se corresponden aproximadamente con los conceptos de proceso e hilo que acabamos de describir. Además, en diversos textos, el término proceso ligero se utiliza como equivalente al término hilo, o un tipo particular de hilo conocido como hilo a nivel del núcleo, o como en el caso de Solaris, a una entidad que mapea hilos a nivel de usuario a hilos a nivel del núcleo.

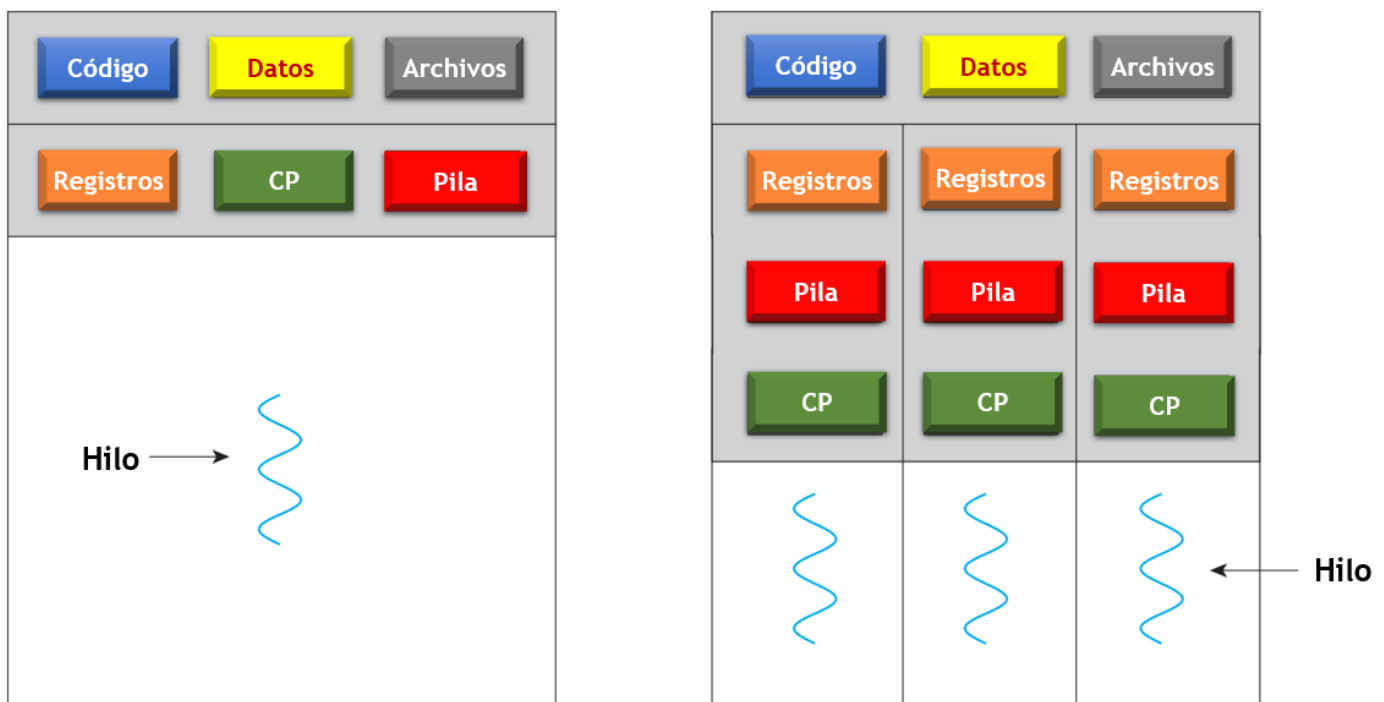


Figura 15: Procesos monohilo y multihilo.

El modelo de proceso visto hasta ahora ha implicado que un proceso es un programa que realiza un único hilo de ejecución. Por ejemplo, cuando un proceso está ejecutando un programa de procesador de textos, se está ejecutando un único hilo de instrucciones. Este único hilo de control permite al proceso realizar sólo una tarea a la vez. Así, el usuario no puede teclear

caracteres y ejecutar el corrector ortográfico simultáneamente. La mayoría de los sistemas operativos modernos han ampliado el concepto de proceso para permitir que un proceso tenga múltiples hilos de ejecución y, por tanto, pueda realizar más de una tarea a la vez. Esta característica es especialmente beneficiosa en los sistemas multinúcleo, donde varios hilos pueden ejecutarse en paralelo. Un procesador de textos multihilo podría, por ejemplo, asignar un hilo para gestionar la entrada del usuario mientras otro hilo ejecuta el corrector ortográfico. En los sistemas que admiten hilos, la PCB se amplía para incluir información sobre cada hilo. También son necesarios otros cambios en todo el sistema para soportar hilos.

En la actualidad, la mayoría de las aplicaciones de software que se ejecutan en los computadores y dispositivos móviles modernos son multihilo. Una aplicación suele implementarse como un proceso independiente con varios hilos de control. Seguidamente, destacamos algunos ejemplos de aplicaciones multihilo:

- Una aplicación que crea miniaturas de fotos a partir de una colección de imágenes puede utilizar un hilo independiente para generar una miniatura de cada imagen por separado.
- Un navegador web puede tener un hilo para mostrar imágenes o texto mientras otro hilo recupera datos de la red.
- Un procesador de textos puede tener un hilo para mostrar los gráficos, otro hilo para responder a las pulsaciones del usuario y un tercer hilo para realizar la revisión ortográfica y gramatical en segundo plano.

Las aplicaciones también pueden diseñarse para aprovechar las capacidades de procesamiento de los sistemas multinúcleo. Estas aplicaciones pueden realizar varias tareas intensivas de la CPU en paralelo a través de los múltiples núcleos del CPU. En determinadas situaciones, una sola aplicación puede tener que realizar varias tareas similares. Por ejemplo, un servidor web acepta solicitudes de clientes para páginas web, imágenes, sonido, etc. Un servidor web ocupado puede tener múltiples clientes accediendo simultáneamente. Si el servidor web se ejecutara como un proceso tradicional de un solo hilo, sólo podría dar servicio a un cliente a la vez, y un cliente podría tener que esperar mucho tiempo para que su solicitud sea atendida.

Una solución es que el servidor se ejecute como un único proceso que acepte peticiones. Cuando el servidor recibe una petición, crea un proceso separado para atenderla. De hecho, este método de creación de procesos era de uso común antes de que los hilos se hicieran populares. Sin embargo, la creación de procesos consume mucho tiempo y recursos. Si el nuevo proceso realizará las mismas tareas que el proceso existente, ¿por qué incurrir en toda esa sobrecarga? Generalmente es más eficiente utilizar un proceso que contenga múltiples hilos. Si el proceso del servidor web es multihilo, el servidor creará un hilo separado que escuche las peticiones de los clientes. Cuando se hace una petición, en lugar de crear otro proceso, el servidor crea un nuevo hilo para atender la petición y reanuda la escucha de peticiones adicionales.

La mayoría de los núcleos de los sistemas operativos también suelen ser multihilo. Por ejemplo, durante el arranque del sistema en los sistemas Linux, se crean varios hilos del kernel. Cada hilo realiza una tarea específica, como la gestión de dispositivos, la gestión de la memoria o el manejo de interrupciones. Muchas aplicaciones también pueden aprovechar las ventajas de los hilos múltiples, incluyendo la ordenación básica, los árboles y los algoritmos de gráficos.

Además, los programadores que deben resolver problemas contemporáneos que requieren un uso intensivo de la CPU en la minería de datos, los gráficos y la inteligencia artificial pueden aprovechar la potencia de los sistemas multinúcleo modernos diseñando soluciones que se ejecuten en paralelo.

Multihilo (Multithreading)

La expresión multihilo se refiere a la capacidad de un sistema operativo para soportar múltiples rutas de ejecución concurrentes dentro de un único proceso. El enfoque tradicional de un solo hilo de ejecución por proceso, en el que no se reconoce el concepto de hilo, se denomina estrategia monohilo. Las dos disposiciones que se muestran en la mitad izquierda de la figura 16 son enfoques de un solo hilo. MS-DOS es un ejemplo de sistema operativo que admite un único proceso de usuario y un único hilo. Otros sistemas operativos, como algunas variantes de UNIX, admiten múltiples procesos de usuario, pero sólo admiten un hilo por proceso. La mitad derecha de la figura 16 muestra los enfoques multihilo. Un entorno de ejecución Java es un ejemplo de sistema de un proceso con múltiples hilos. Lo que interesa aquí es el uso de múltiples procesos, cada uno de los cuales soporta múltiples hilos. Este enfoque se adopta en Windows, Solaris y muchas versiones modernas de UNIX, entre otros. Veamos ahora una descripción general de los multihilos.

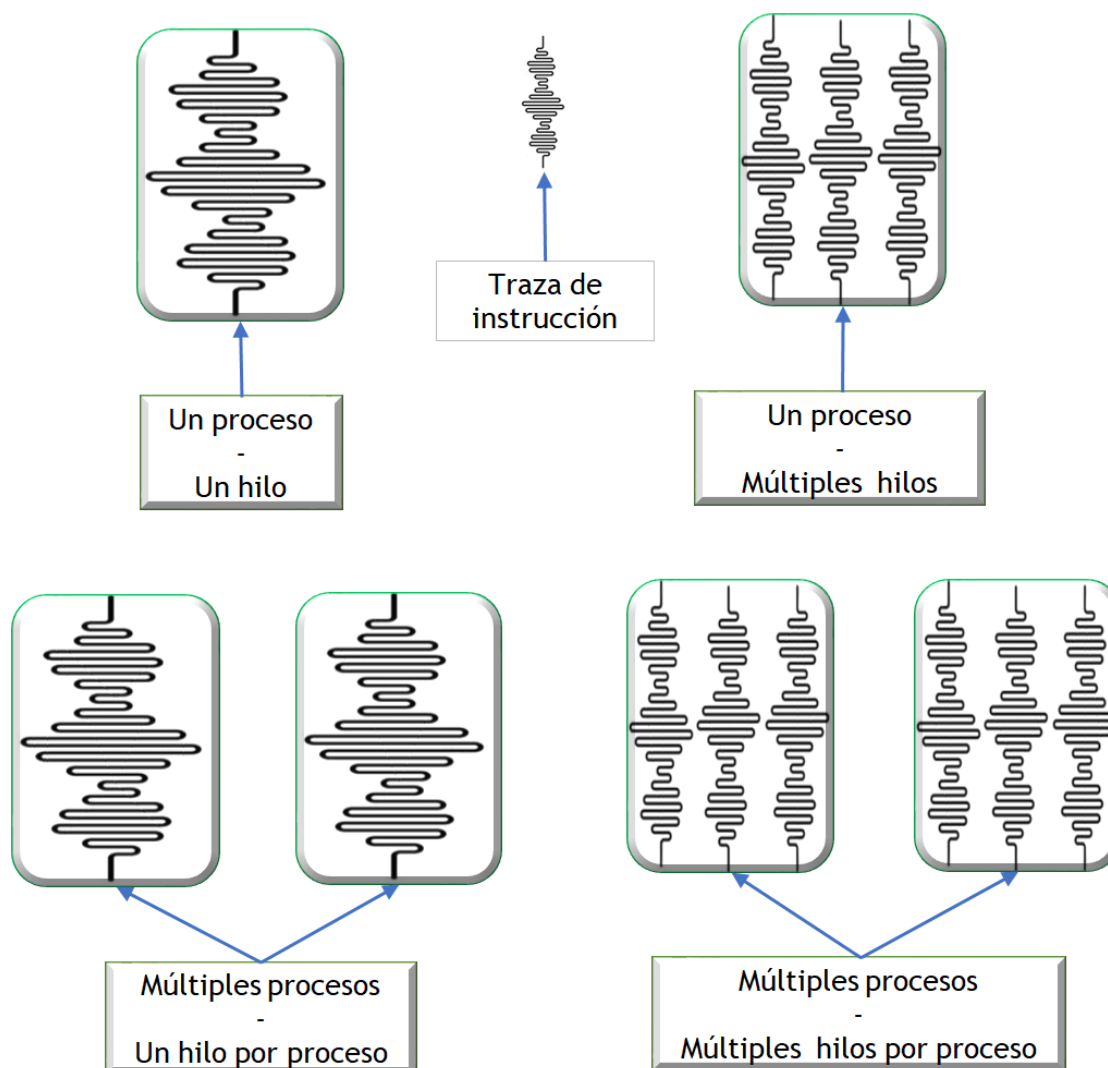


Figura 16: Hilos y procesos.

En un entorno multihilo, un proceso se define como la unidad de asignación de recursos y una unidad de protección. A los procesos se les asocia lo siguiente:

- Un espacio de direcciones virtual que contiene la imagen del proceso.
- Acceso protegido a los procesadores, a otros procesos (para la comunicación entre procesos), a los archivos y a los recursos de E/S (dispositivos y canales).

Dentro de un proceso, puede haber uno o más hilos, cada uno con lo siguiente:

- Un estado de ejecución de hilos (En ejecución, Listo, etc.)
- Un contexto de hilo guardado cuando no se está ejecutando; una forma de ver un hilo es como un contador de programa independiente que opera dentro de un proceso.
- Una pila de ejecución.
- Algún tipo de almacenamiento estático por hilo para las variables locales.
- Acceso a la memoria y los recursos de su proceso, compartido con todos los demás hilos de ese proceso.

Luego, los sistemas operativos que implementan multithreading deberán manejar 2 estructuras: el bloque de control de procesos (BCP) que contiene información sobre el espacio de direcciones, asignación de recursos, permisos de acceso y valores default para sus threads y el bloque de control de hebras (BCH) que contiene información de estado de la hebra e información de scheduling: identificación, estado, registros del procesador CPU, prioridad dinámica, prioridad base, tiempo acumulado de ejecución en modo usuario y supervisor, etc.

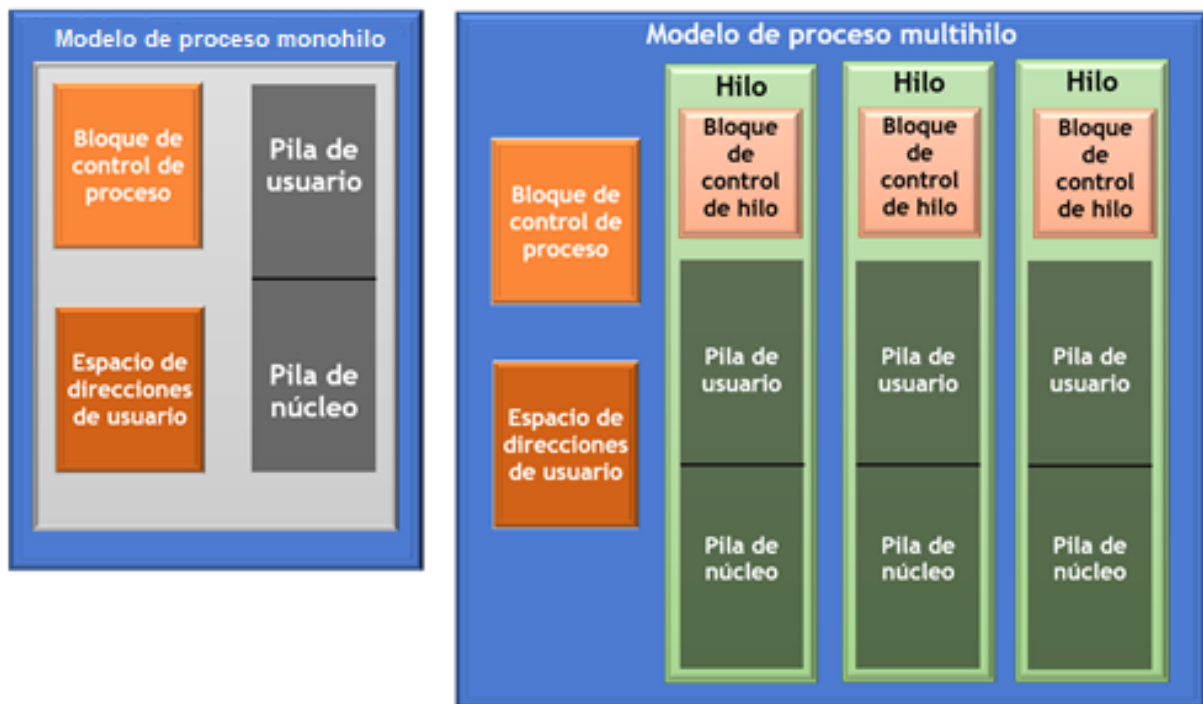


Figura 17: Modelos de proceso con un único hilo y multihilo.

La figura 17 ilustra la distinción entre hilos y procesos desde el punto de vista de la gestión de procesos. En un modelo de proceso de un solo hilo (es decir, no hay un concepto distinto de hilo), la representación de un proceso incluye su bloque de control de proceso y el espacio de

direcciones del usuario, así como las pilas del usuario y del núcleo para gestionar el comportamiento de llamada/retorno de la ejecución del proceso. Mientras el proceso se ejecuta, controla los registros del procesador. El contenido de estos registros se guarda cuando el proceso no se está ejecutando. En un entorno multihilo, sigue habiendo un único bloque de control del proceso y un espacio de direcciones de usuario asociado al proceso, pero ahora hay pilas separadas para cada hilo, así como un bloque de control separado para cada hilo que contiene valores de registro, prioridad y otra información de estado relacionada con el hilo.

Así, todos los hilos de un proceso comparten el estado y los recursos de ese proceso. Residen en el mismo espacio de direcciones y tienen acceso a los mismos datos. Cuando un hilo altera un elemento de datos en la memoria, otros hilos ven los resultados cuando acceden a ese elemento. Si un hilo abre un archivo con privilegios de lectura, otros hilos en el mismo proceso también pueden leer de ese archivo.

Las principales ventajas de los hilos se derivan de las implicaciones de rendimiento:

- Se tarda mucho menos en crear un nuevo hilo en un proceso existente que en crear un proceso nuevo. Los estudios realizados por los desarrolladores de sistemas operativos muestran que la creación de hilos es diez veces más rápida que la creación de procesos.
- Se necesita menos tiempo para terminar un hilo que un proceso.
- Se necesita menos tiempo para cambiar entre dos hilos dentro del mismo proceso que para cambiar entre procesos.
- Los hilos mejoran la eficiencia en la comunicación entre diferentes programas en ejecución. En la mayoría de los sistemas operativos, la comunicación entre procesos independientes requiere la intervención del núcleo para proporcionar la protección y los mecanismos necesarios para la comunicación. Sin embargo, como los hilos de un mismo proceso comparten memoria y archivos, pueden comunicarse entre sí sin necesidad de invocar al núcleo.

Por lo tanto, si hay una aplicación o función que debe ser implementada como un conjunto de unidades de ejecución relacionadas, es mucho más eficiente hacerlo como una colección de hilos en lugar de una colección de procesos separados.

Un ejemplo de una aplicación que podría hacer uso de hilos es un servidor de archivos. Cada vez que llega una nueva solicitud de archivo, se puede generar un nuevo hilo para el programa de gestión de archivos. Dado que un servidor manejará muchas solicitudes, se crearán y destruirán muchos hilos en un período corto. Si el servidor se ejecuta en un equipo multiprocesador, entonces múltiples hilos dentro del mismo proceso pueden estar ejecutándose simultáneamente en diferentes procesadores. Además, dado que los procesos o hilos en un servidor de archivos deben compartir los datos de los archivos y, por tanto, coordinar sus acciones, es más rápido utilizar hilos y memoria compartida que procesos y paso de mensajes para esta coordinación.

La construcción de hilos también es útil en un solo procesador para simplificar la estructura de un programa que lógicamente está haciendo varias funciones diferentes. Podemos citar cuatro ejemplos de los usos de los hilos en un sistema de multiprocesamiento de un solo usuario:

- **Trabajo en primer y segundo plano.** Por ejemplo, en un programa de hoja de cálculo, un hilo podría mostrar los menús y leer la entrada del usuario, mientras que otro hilo ejecuta los comandos del usuario y actualiza la hoja de cálculo. Esta disposición suele aumentar la velocidad percibida de la aplicación, ya que permite que el programa solicite el siguiente comando antes de que se complete el anterior.
- **Procesamiento asíncrono.** Los elementos asíncronos del programa pueden implementarse como hilos. Por ejemplo, como protección contra los fallos de alimentación, se puede diseñar un procesador de textos que escriba su memoria de acceso aleatorio (RAM) en el disco una vez cada minuto. Se puede crear un hilo cuyo único trabajo sea la copia de seguridad periódica y que se programe directamente con el sistema operativo; no es necesario un código especial en el programa principal para prever comprobaciones de tiempo o coordinar la entrada y la salida.
- **Velocidad de ejecución.** Un proceso multihilo puede calcular un lote de datos mientras lee el siguiente lote de un dispositivo. En un sistema multiprocesador, varios hilos del mismo proceso pueden ejecutarse simultáneamente. Así, aunque un hilo esté bloqueado por una operación de E/S para leer un lote de datos, otro hilo puede estar ejecutándose.
- **Estructura modular del programa.** Los programas que implican una variedad de actividades o una variedad de fuentes y destinos de entrada y salida pueden ser más fáciles de diseñar e implementar utilizando hilos.

En un sistema operativo que admite hilos, la programación y el envío se realizan por hilos; por lo tanto, la mayor parte de la información de estado relativa a la ejecución se mantiene en estructuras de datos a nivel de hilo. Sin embargo, hay varias acciones que afectan a todos los hilos de un proceso y que el sistema operativo debe gestionar a nivel de proceso. Por ejemplo, la suspensión implica intercambiar el espacio de direcciones de un proceso fuera de la memoria principal para dejar espacio al espacio de direcciones de otro proceso. Como todos los hilos de un proceso comparten el mismo espacio de direcciones, todos los hilos se suspenden al mismo tiempo. Del mismo modo, la terminación de un proceso termina todos los hilos dentro de ese proceso.

Las ventajas de la programación multihilo pueden desglosarse en cuatro grandes categorías:

- **Capacidad de respuesta.** El multihilo de una aplicación interactiva puede permitir que un programa siga ejecutándose, aunque parte de él esté bloqueado o esté realizando una operación larga, lo que aumenta la capacidad de respuesta al usuario. Esta cualidad es especialmente útil en el diseño de interfaces de usuario. Por ejemplo, considere lo que ocurre cuando un usuario hace clic en un botón que da lugar a la realización de una operación que requiere mucho tiempo. Una aplicación de un solo hilo no respondería ante el usuario hasta que la operación se hubiera completado. En cambio, si la operación que consume tiempo se realiza en un hilo separado y asíncrono, la aplicación sigue respondiendo al usuario.
- **Compartir recursos.** Los procesos pueden compartir recursos sólo a través de técnicas como la memoria compartida y el paso de mensajes. Estas técnicas deben ser organizadas explícitamente por el programador. Sin embargo, los hilos comparten por defecto la memoria y los recursos del proceso al que pertenecen. La ventaja de compartir código y

datos es que permite que una aplicación tenga varios hilos de actividad diferentes dentro del mismo espacio de direcciones.

- **Economía.** La asignación de memoria y recursos para la creación de procesos es costosa. Dado que los hilos comparten los recursos del proceso al que pertenecen, es más económico crear y cambiar de contexto los hilos. Puede ser difícil medir empíricamente la diferencia en la sobrecarga, pero en general la creación de hilos consume menos tiempo y memoria que la creación de procesos. Además, el cambio de contexto suele ser más rápido entre hilos que entre procesos.
- **Escalabilidad.** Las ventajas del multihilo pueden ser aún mayores en una arquitectura multiprocesador, donde los hilos pueden ejecutarse en paralelo en diferentes núcleos de procesamiento. Un proceso de un solo hilo puede ejecutarse en un solo procesador, independientemente de cuántos estén disponibles.

Funcionalidades de los hilos

Al igual que los procesos, los hilos tienen estados de ejecución y pueden sincronizarse entre sí. Veamos estos dos aspectos de la funcionalidad de los hilos por separado.

Estados de los hilos

Al igual que con los procesos, los principales estados de un hilo son: Ejecución, Listo y Bloqueado. Por lo general, no tiene sentido asociar los estados de suspensión con los hilos porque dichos estados son conceptos a nivel de proceso. Si un proceso es intercambiado, todos sus hilos son necesariamente intercambiados porque todos comparten el espacio de direcciones del proceso. Hay cuatro operaciones básicas de hilos asociadas a un cambio de estado de hilos:

- **Generación:** Normalmente, cuando se crea un nuevo proceso, también se crea un subproceso para ese proceso. Posteriormente, un hilo dentro de un proceso puede engendrar otro hilo dentro del mismo proceso, proporcionando un puntero de instrucción y argumentos para el nuevo hilo. El nuevo hilo recibe su propio contexto de registro y espacio de pila y se coloca en la cola de espera.
- **Bloqueo:** Cuando un hilo necesita esperar un evento, se bloqueará (guardando sus registros de usuario, contador de programa y punteros de pila). El procesador puede ahora pasar a la ejecución de otro hilo listo en el mismo proceso o en uno diferente.
- **Desbloqueo:** Cuando se produce el evento por el que un hilo está bloqueado, el hilo se mueve a la cola de listos.
- **Finalizar:** Cuando un hilo finaliza, su contexto de registro y sus pilas se desasignan.

Analicemos las ventajas de rendimiento de los hilos que no bloquean un proceso entero. Consideremos un programa que realiza dos llamadas a procedimientos remotos a dos hosts diferentes para obtener un resultado combinado. En un programa de un solo hilo, los resultados se obtienen en secuencia, por lo que el programa debe esperar una respuesta de cada servidor por turno. Reescribiendo el programa para usar un hilo separado para cada llamada a procedimiento remoto se consigue un aumento sustancial de la velocidad. Tengamos en cuenta que, si este programa opera en un monoprocesador, las peticiones deben ser generadas

secuencialmente y los resultados procesados en secuencia; sin embargo, el programa espera concurrentemente las dos respuestas.

Sincronización de hilos

Todos los hilos de un proceso comparten el mismo espacio de direcciones y otros recursos, como los archivos abiertos. Cualquier alteración de un recurso por parte de un hilo afecta al entorno de los demás hilos del mismo proceso. Por tanto, es necesario sincronizar las actividades de los distintos hilos para que no interfieran entre sí ni corrompan las estructuras de datos. Por ejemplo, si dos hilos intentan añadir cada uno un elemento a una lista doblemente enlazada al mismo tiempo puede perderse un elemento o la lista puede acabar malformada. Los problemas que se plantean y las técnicas utilizadas en la sincronización de hilos son, en general, los mismos que para la sincronización de procesos que serán analizadas más adelante.

Implementación de hilos

Nuestra discusión hasta ahora ha tratado los hilos en un sentido genérico. Sin embargo, existen dos grandes categorías de implementación de hilos: los *hilos a nivel de usuario* (ULT: User Level Threads) y los *hilos a nivel del núcleo* (KLT: Kernel Level Threads). Estos últimos también se conocen en la literatura como hilos soportados por el núcleo o procesos ligeros. En la primera clase el soporte para los hilos es proporcionado a nivel de usuario, mientras que en la segunda clase el soporte para los hilos es proporcionado a nivel del núcleo. Los hilos de usuario se soportan por encima del núcleo y se gestionan sin soporte del núcleo, mientras que los hilos del núcleo son soportados y gestionados directamente por el sistema operativo. Prácticamente todos los sistemas operativos contemporáneos, incluyendo Windows, Linux y macOS, soportan hilos del núcleo. En última instancia, debe existir una relación entre los hilos de usuario y los hilos del núcleo, por lo que veremos tres formas comunes de establecer dicha relación: el modelo de muchos a uno, el modelo de uno a uno y el modelo de muchos a muchos.

Hilos a nivel de usuario (ULT)

En una instalación ULT pura, todo el trabajo de gestión de hilos lo realiza la aplicación y el núcleo no conoce la existencia de hilos. Cualquier aplicación puede ser programada para ser multihilo utilizando una biblioteca de hilos, que es un paquete de rutinas para la gestión de ULT. La biblioteca de hilos contiene código para crear y destruir hilos, para pasar mensajes y datos entre hilos, para programar la ejecución de hilos, y para guardar y restaurar contextos de hilos.

Por defecto, una aplicación comienza con un único hilo y empieza a ejecutarse en ese hilo. Esta aplicación y su hilo se asignan a un único proceso gestionado por el kernel. En cualquier momento en que la aplicación se esté ejecutando (el proceso está en el estado de ejecución), la aplicación puede generar un nuevo hilo que se ejecute dentro del mismo proceso. La creación se realiza invocando la utilidad de creación en la biblioteca de hilos. El control se pasa a esa utilidad mediante una llamada a un procedimiento. La biblioteca de hilos crea una estructura de datos para el nuevo hilo y luego pasa el control a uno de los hilos dentro de este proceso que está en el estado Listo, utilizando algún algoritmo de programación. Cuando se pasa el control a la biblioteca, se guarda el contexto del hilo actual, y cuando se pasa el control de la biblioteca

a un hilo, se restaura el contexto de ese hilo. El contexto consiste esencialmente en el contenido de los registros de usuario, el contador de programa y los punteros de la pila.

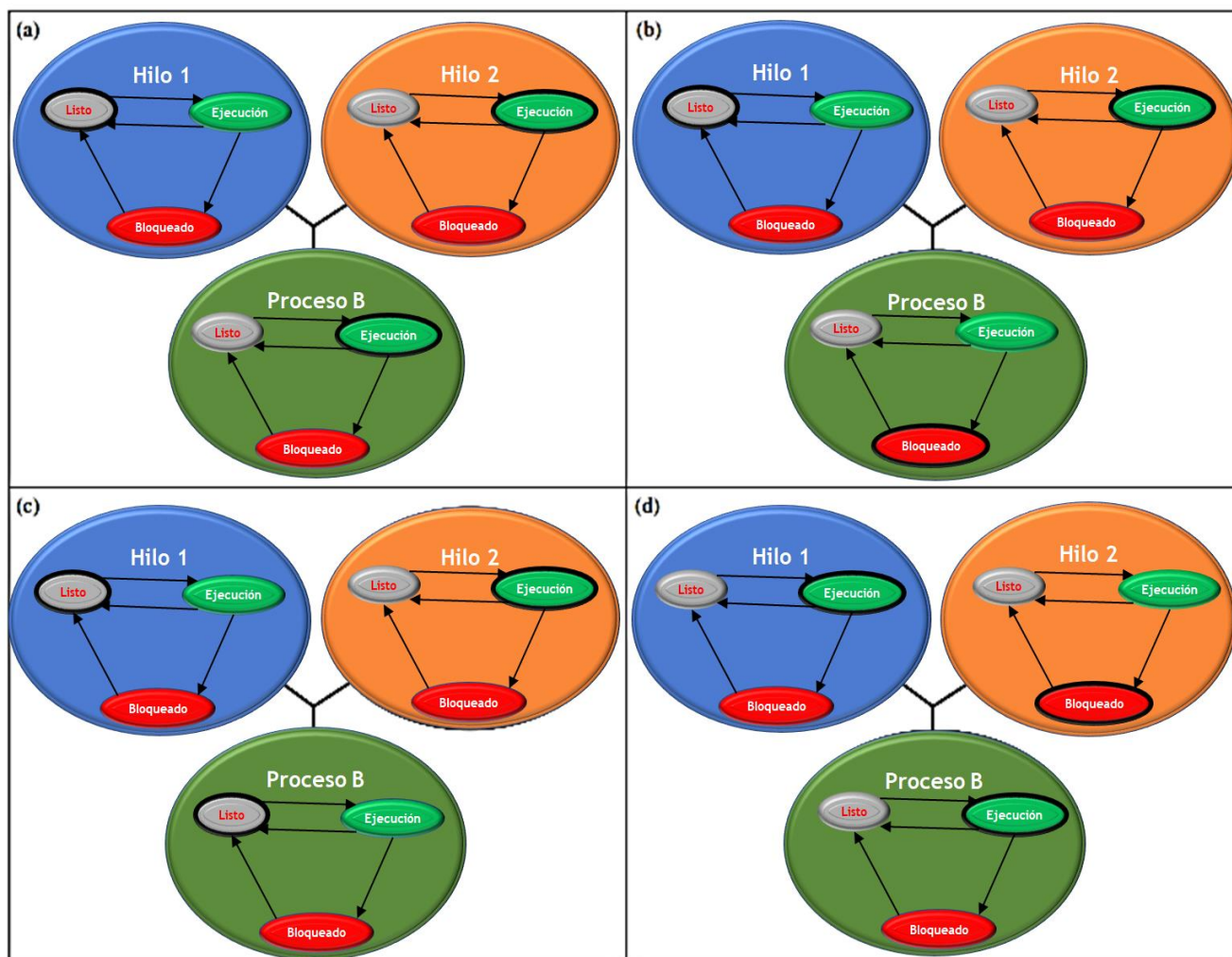


Figura 18: Ejemplos de las relaciones entre los estados de los hilos a nivel de usuario y los estados del proceso. Toda la actividad descrita en el párrafo anterior tiene lugar en el espacio de usuario y dentro de un único proceso. El núcleo no es consciente de esta actividad. El kernel continúa programando el proceso como una unidad y asigna un único estado de ejecución (Listo, En ejecución, Bloqueado, etc.) a ese proceso. Los siguientes ejemplos deberían aclarar la relación entre la programación de hilos y la programación de procesos. Supongamos que el proceso B se está ejecutando en su hilo 2; los estados del proceso y dos ULT's que forman parte del proceso se muestran en la Figura 18a. Cada una de las siguientes es una posible ocurrencia:

- **Caso 1.** La aplicación que se ejecuta en el hilo 2 realiza una llamada al sistema que bloquea a B. Por ejemplo, se realiza una llamada de E/S. Esto hace que el control se transfiera al kernel. El kernel invoca la acción de E/S, coloca al proceso B en el estado de Bloqueo, y cambia a otro proceso. Mientras tanto, según la estructura de datos mantenida por la biblioteca de hilos, el hilo 2 del proceso B sigue en el estado de Ejecución. Es importante notar que el hilo 2 no está realmente en ejecución en el sentido de ser ejecutado en un procesador; pero es percibido como estando en el estado de Ejecución por la biblioteca del hilo. Los diagramas de estado correspondientes se muestran en la figura 18b.

- **Caso 2.** Una interrupción del reloj pasa el control al núcleo, y éste determina que el proceso que se está ejecutando actualmente (B) ha agotado su franja de tiempo. El núcleo coloca el proceso B en el estado Listo y cambia a otro proceso. Mientras tanto, según la estructura de datos mantenida por la biblioteca de hilos, el hilo 2 del proceso B sigue en el estado de Ejecución. Los diagramas de estado correspondientes se muestran en la figura 18c.
- **Caso 3.** El hilo 2 ha llegado a un punto en el que necesita alguna acción realizada por el hilo 1 del proceso B. El hilo 2 entra en un estado de Bloqueo y el hilo 1 pasa de Listo a En Ejecución. El proceso en sí mismo permanece en el estado de Ejecución. Los diagramas de estado correspondientes se muestran en la figura 18d.

En los casos 1 y 2 (Figuras 18b y 18c), cuando el núcleo devuelve el control al proceso B, la ejecución se reanuda en el hilo 2. También hay que tener en cuenta que un proceso puede ser interrumpido, ya sea por agotar su franja de tiempo o por ser adelantado por un proceso de mayor prioridad, mientras está ejecutando código en la biblioteca del hilo. Por lo tanto, un proceso puede estar en medio de un cambio de hilo, pasando de un hilo a otro, cuando se interrumpe. Cuando ese proceso se reanuda, la ejecución continúa dentro de la biblioteca del hilo, que completa el cambio de hilo y transfiere el control a otro hilo dentro de ese proceso.

El uso de ULT's en lugar de KLT's tiene algunas ventajas, entre ellas:

- La conmutación de hilos no requiere privilegios del modo kernel porque todas las estructuras de datos de gestión de hilos están dentro del espacio de direcciones de usuario de un único proceso. Por lo tanto, el proceso no cambia al modo kernel para realizar la gestión de hilos. Esto ahorra la sobrecarga de dos cambios de modo (de usuario a kernel; de kernel a usuario).
- La planificación puede ser específica de la aplicación. Una aplicación puede beneficiarse de un simple algoritmo de planificación Round-Robin, mientras que otra puede beneficiarse de un algoritmo de planificación basado en la prioridad. El algoritmo de planificación puede adaptarse a la aplicación sin alterar el planificador del sistema operativo subyacente.
- Los ULT pueden funcionar en cualquier sistema operativo. No es necesario realizar cambios en el núcleo subyacente para soportar los ULT's. La biblioteca de hilos es un conjunto de funciones a nivel de aplicación que comparten todas las aplicaciones.

Sin embargo, existen dos desventajas claras de los ULT's en comparación con los KLT's:

- En un sistema operativo típico, muchas llamadas al sistema son de bloqueo. Como resultado, cuando un ULT ejecuta una llamada al sistema, no sólo se bloquea ese hilo, sino que también se bloquean todos los hilos dentro del proceso.
- En una estrategia ULT pura, una aplicación multihilo no puede aprovechar el multiprocesamiento. Un kernel asigna un proceso a un solo procesador a la vez. Por lo tanto, sólo un único hilo dentro de un proceso puede ejecutarse a la vez. En efecto, tenemos multiprogramación a nivel de aplicación dentro de un único proceso. Mientras que esta multiprogramación puede resultar en una aceleración significativa de la

aplicación, hay aplicaciones que se beneficiarían de la capacidad de ejecutar porciones de código simultáneamente.

Hay formas de solucionar estos dos problemas. Por ejemplo, ambos problemas pueden superarse escribiendo una aplicación como múltiples procesos en lugar de múltiples hilos. Pero este enfoque elimina la principal ventaja de los hilos: Cada interruptor se convierte en un interruptor de proceso en lugar de un interruptor de hilo, lo que resulta en una sobrecarga mucho mayor.

Otra forma de superar el problema de los hilos bloqueantes es utilizar una técnica conocida como *jacketing* o *revestimiento*. El propósito del jacketing es convertir una llamada al sistema que se bloquea en una llamada al sistema que no se bloquea. Por ejemplo, en lugar de llamar directamente a una rutina de E/S del sistema, un hilo llama a una *rutina jacket* de E/S a nivel de aplicación. Dentro de esta rutina jacket hay un código que comprueba si el dispositivo de E/S está ocupado. Si lo está, el hilo entra en el estado de Bloqueo y pasa el control (a través de la biblioteca de hilos) a otro hilo. Cuando este hilo recibe de nuevo el control, la rutina jacket comprueba de nuevo el dispositivo de E/S.

Hilos a nivel del núcleo (KLT)

En una instalación KLT pura, todo el trabajo de gestión de hilos lo realiza el núcleo. No hay código de gestión de hilos en el nivel de aplicación, simplemente una interfaz de programación de aplicaciones (API) para la instalación de hilos del núcleo. Windows es un ejemplo de este enfoque. El núcleo mantiene la información de contexto para el proceso en su conjunto y para los hilos individuales dentro del proceso. La planificación por parte del núcleo se realiza por hilos. Este enfoque supera los dos principales inconvenientes del enfoque ULT. En primer lugar, el núcleo puede planificar simultáneamente varios hilos del mismo proceso en varios procesadores. En segundo lugar, si un hilo de un proceso se bloquea, el núcleo puede seleccionar otro hilo del mismo proceso. Otra ventaja del enfoque KLT es que las propias rutinas del núcleo pueden ser multihilo.

Sin embargo, la principal desventaja del enfoque KLT en comparación con el enfoque ULT es que la transferencia de control de un hilo a otro dentro del mismo proceso requiere un cambio de modo en el núcleo. En distintas investigaciones se comprobó que, mientras hay un aumento significativo de la velocidad mediante el uso de KLT multithreading en comparación con los procesos de un solo hilo, también hay un aumento adicional significativo de la velocidad mediante el uso de ULT's. Sin embargo, el hecho de que el aumento de velocidad se produzca o no depende de la naturaleza de las aplicaciones. Si la mayoría de los cambios de hilos en una aplicación requieren acceso al modo kernel, entonces un esquema basado en ULT puede no funcionar mucho mejor que un esquema basado en KLT.

Enfoques combinados

Algunos sistemas operativos proporcionan una facilidad combinada ULT/KLT. En un sistema combinado, la creación de hilos se realiza completamente en el espacio de usuario, al igual que la mayor parte de la programación y sincronización de hilos dentro de una aplicación. Los múltiples ULT's de una sola aplicación se asignan a un número (menor o igual) de KLT's. El programador puede ajustar el número de KLT's para un procesador y aplicación en particular, para conseguir los mejores resultados posibles.

En un enfoque combinado, múltiples hilos dentro de la misma aplicación pueden ejecutarse en paralelo en múltiples procesadores, y una llamada bloqueante al sistema no necesita bloquear todo el proceso. Si se diseña adecuadamente, este enfoque debería combinar las ventajas de los enfoques ULT y KLT puros y minimizar las desventajas. Solaris es un buen ejemplo de un sistema operativo que utiliza este enfoque combinado. La versión actual de Solaris limita la relación ULT/KLT a uno.

Otras configuraciones

Como hemos dicho, los conceptos de asignación de recursos y unidad de despacho se han plasmado tradicionalmente en el concepto único de proceso, es decir, como una relación 1 a 1 entre hilos y procesos. Recientemente, ha habido mucho interés en prever múltiples hilos dentro de un único proceso, que es una relación de muchos a uno. Sin embargo, como muestra la tabla *Relación entre hilos y procesos*, también se investigaron las otras dos combinaciones, a saber, una relación de muchos a muchos y una relación de uno a muchos.

Relación entre hilos y procesos		
Hilos : Procesos	Descripción	Sistemas de ejemplo
1 : 1	Cada hilo de ejecución es un único proceso con su propio espacio de direcciones y recursos.	Windows, Linux, Implementaciones UNIX tradicionales.
M : 1	Un proceso define un espacio de direcciones y pertenencia dinámica de recursos. Se pueden crear y ejecutar múltiples hilos en ese proceso.	Windows NT, Solaris, Linux, OS/2, OS/390, MACH.
1 : M	Un hilo puede migrar de un entorno de proceso a otro. Esto permite a los hilos moverse fácilmente entre distintos sistemas.	Ra (Clouds), Emerald.
M : M	Combina atributos de M:1 y casos 1:M.	TRIX.

Modelo de relación uno a uno

El modelo uno a uno asigna cada hilo de usuario a un hilo del núcleo. Proporciona más concurrencia que el modelo muchos-a-uno al permitir que otro hilo se ejecute cuando un hilo hace una llamada al sistema de bloqueo. También permite que varios hilos se ejecuten en paralelo en los multiprocesadores. La única desventaja de este modelo es que la creación de un hilo de usuario requiere la creación del correspondiente hilo del núcleo, y muchos hilos del núcleo pueden sobrecargar el rendimiento de un sistema. Linux, junto con la familia de sistemas operativos Windows, implementan el modelo uno a uno.

Modelo de relación muchos a uno

El modelo de muchos a uno asigna muchos hilos a nivel de usuario a un hilo del núcleo. La gestión de los hilos la realiza la biblioteca de hilos en el espacio de usuario, por lo que es eficiente. Sin embargo, todo el proceso se bloqueará si un hilo hace una llamada al sistema que se bloquea. Además, como sólo un hilo puede acceder al núcleo a la vez, varios hilos no pueden

ejecutarse en paralelo en sistemas multinúcleo. Green threads, una biblioteca de hilos disponible para los sistemas Solaris y adoptada en las primeras versiones de Java, utilizaba el modelo de muchos a uno. Sin embargo, muy pocos sistemas siguen utilizando este modelo debido a su incapacidad para aprovechar los múltiples núcleos de procesamiento, que ahora se han convertido en un estándar en la mayoría de los sistemas informáticos.

Modelo de relación uno a muchos

En el campo de los sistemas operativos distribuidos (diseñados para controlar sistemas informáticos distribuidos), ha habido interés en el concepto de hilo como entidad que puede moverse entre espacios de direcciones. Un ejemplo notable de esta investigación es el sistema operativo Clouds, y especialmente su núcleo, conocido como Ra. Otro ejemplo es el sistema Emerald.

Un hilo en Clouds es una unidad de actividad, desde la perspectiva del usuario. Un proceso es un espacio de direcciones virtual con un bloque de control de proceso asociado. Al crearse, un hilo comienza a ejecutarse en un proceso invocando un punto de entrada a un programa en ese proceso. Los hilos pueden moverse de un espacio de direcciones a otro, y de hecho atraviesan los límites del computador (es decir, se mueven de un computador a otro). Cuando un hilo se desplaza, debe llevar consigo cierta información, como la terminal de control, los parámetros globales y la guía de programación (por ejemplo, la prioridad).

El enfoque de Clouds proporciona una forma eficaz de aislar tanto a los usuarios como a los programadores de los detalles del entorno distribuido. La actividad de un usuario puede representarse como un único hilo, y el movimiento de ese hilo entre computadores puede ser dictado por el sistema operativo por una serie de razones relacionadas con el sistema, tal como la necesidad de acceder a un recurso remoto o el equilibrado de carga.

Modelo de relación muchos a muchos

La idea de tener una relación de muchos a muchos entre hilos y procesos se ha explorado en el sistema operativo experimental TRIX. En TRIX, existen los conceptos de dominio e hilo. Un dominio es una entidad estática, que consiste en un espacio de direcciones y "puertos" a través de los cuales se pueden enviar y recibir mensajes. Un hilo es una ruta de ejecución única, con una pila de ejecución, estado del procesador e información de programación.

Al igual que con los enfoques de multihilo discutidos hasta ahora, múltiples hilos pueden ejecutarse en un solo dominio, proporcionando las ganancias de eficiencia discutidas anteriormente. Sin embargo, también es posible que una actividad de un solo usuario, o una aplicación, se ejecute en múltiples dominios. En este caso, existe un hilo que puede pasar de un dominio a otro.

El uso de un único hilo en múltiples dominios parece motivado principalmente por el deseo de proporcionar herramientas de estructuración al programador. Por ejemplo, considere un programa que hace uso de un subprograma de E/S. En un entorno de multiprogramación que permita procesos creados por el usuario, el programa principal podría generar un nuevo proceso para manejar la E/S, y luego continuar la ejecución. Sin embargo, si el progreso futuro del programa principal depende del resultado de la operación de E/S, entonces el programa

principal tendrá que esperar a que el otro programa de E/S termine. Hay varias maneras de implementar esta aplicación:

1. El programa completo puede ser implementado como un solo proceso. Esta es una solución razonable y directa. Existen inconvenientes relacionados con la gestión de la memoria. El proceso, como un todo, puede requerir una memoria principal considerable para ejecutarse eficientemente, mientras que el subprograma de E/S requiere un espacio de direcciones relativamente pequeño para amortiguar la E/S y para manejar la cantidad relativamente pequeña de código del programa. Debido a que el programa de E/S se ejecuta en el espacio de direcciones del programa más grande, o bien todo el proceso debe permanecer en la memoria principal durante la operación de E/S, o la operación de E/S está sujeta a intercambio. Este efecto de gestión de memoria también existiría si el programa principal y el subprograma de E/S se implementaran como dos hilos en el mismo espacio de direcciones.
2. El programa principal y el subprograma de E/S pueden ser implementados como dos procesos separados. Esto incurre en la sobrecarga de crear el proceso subordinado. Si la actividad de E/S es frecuente, hay que dejar vivo el proceso subordinado, lo que consume recursos de gestión, o crear y destruir frecuentemente el subprograma, lo que es ineficiente.
3. Tratar el programa principal y el subprograma de E/S como una sola actividad que debe ser implementada como un solo hilo. Sin embargo, se podría crear un espacio de direcciones (dominio) para el programa principal y otro para el subprograma de E/S. Así, el hilo puede moverse entre los dos espacios de direcciones a medida que la ejecución avanza. El sistema operativo puede gestionar los dos espacios de direcciones de forma independiente, y no se incurre en una sobrecarga de creación de procesos. Además, el espacio de direcciones utilizado por el subprograma de E/S también podría ser compartido por otros programas simples de E/S.

Las experiencias de los desarrolladores de TRIX indican que la tercera opción tiene mérito, y puede ser la solución más eficaz para algunas aplicaciones.

Bibliografía de referencia del texto e imágenes

- Arpaci-Dusseau, R., Arpaci-Dusseau, A., & Reiher, P. (2018). *Operating Systems: Three Easy Pieces 1.00 Edición*. Los Ángeles, California: CreateSpace Independent Publishing Platform.
- Carretero Pérez, J., De Miguel Anasagasti, P., García Carballeira, F., & Pérez Costoya, F. (2007). *Sistemas Operativos: Una visión aplicada, Segunda Edición*. Buenos Aires: Mc. Graw Hill.
- Carretero Perez, J., Garcia Carballeira, F., & Perez Costoya, F. (2021). *Sistemas Operativos: Una Visión Aplicada Tercera Edición Volumen I*. Independently Published.
- Carretero Perez, J., Garcia Carballeira, F., & Perez Costoya, F. (2021). *Sistemas Operativos: Una Visión Aplicada, Tercera Edición, Volumen II*. Independently Published.
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts, Tenth Edition*. Hoboken, New Jersey: John Wiley & Sons, Inc.
- Silva, M. (2015). *Sistemas Operativos*. Ciudad Autónoma de Buenos Aires: Alfaomega Grupo Editor.
- Stallings, W. (2012). *Operating Systems, Internals and Design Principles, Seventh Edition*. Upper Saddle River; New Jersey: Pearson Education, Inc.
- Stallings, W. (2018). *Operating Systems: Internals and Design Principles, 9th Edition*. Upper Saddle River; New Jersey: Pearson Education, Inc.
- Stuart, B. L. (2021). *Principles of Operating Systems: Design and Applications*. Morrisville, North Carolina: Lulu.com.
- Tanenbaum, A. S. (2009). *Sistemas operativos modernos Tercera edición*. México: Pearson/Prentice Hall.
- Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems, Fourth Edition*. Upper Saddle River, New Jersey: Pearson Education, Inc.
- Tanenbaum, A. S., & Bos, H. (2023). *Modern Operating Systems, 5th edition*. Upper Saddle River, New Jersey: Pearson Education, Inc.
- Wolf, G. (2015). *Fundamentos de sistemas operativos*. México: Instituto de Investigaciones Económicas UNAM.